2025年终必看：开源大模型哪家强？9大模型架构演变历程一次性看明白

原创 腾讯程序员 腾讯技术工程 2025年11月10日 01:43 广东 109人



当大模型开始“进化论”——一场静默却颠覆认知的技术跃迁。2025年已即将结束，开源大模型的演进速度，是否已让你感到“迭代焦虑”？半年多前我们还在解析DeepSeek的架构设计，而今天，新一轮模型爆发已重新定义竞争的边界。国庆前各大模型争相更新也是让人感受到领域的卷度。

当我们看到Claude在复杂逻辑推理中展现近乎人类的连贯性，当Kimi-K2轻松处理数百万字的上下文而不“失焦”，当越来越多企业用开源模型跑通了过去只有闭源API才能支撑的业务场景——一个问题自然浮现：在这静默却汹涌的半年里，大模型究竟“进化”了什么？

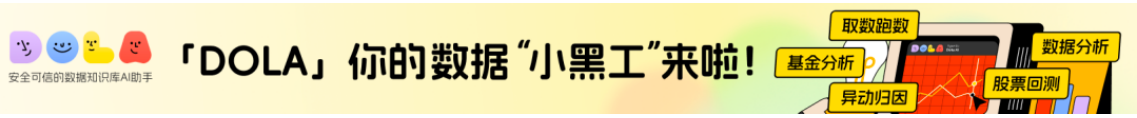
这不是一次简单的参数刷新或基准测试分数的提升，而是一场由内而外的“能力质变”。模型开始从“回答者”蜕变为“思考者”，从单一语言处理器演进为多模态任务的“协调中枢”。更重要的是，效率与性能的兼得正在成为现实：稀疏激活、MoE架构、动态计算路径……这些技术让百亿参数模型可在消费级GPU上流畅对话。

如果你好奇这背后的技术主线如何串联，如果你希望理解模型能力跃迁背后的根本动因，那么本文正是为你准备的。我们将一同揭开大模型演进的核心逻辑。

模型名称	核心点	创新点	与其他模型的不同点
DeepSeekV3/R1	6710亿总参数，采用MoE架构，推理时仅激活370亿参数；使用MLA替代GQA	1.多头潜在注意力（MLA）：压缩键值张量至低维空间存储，提升性能且减少KV缓存内存；2.MoE架构含共享专家，始终活跃，助力学习通用模式与专业模式	1.注意力机制上，用MLA而非GQA，且MLA性能优于MHA和GQA2.MoE架构中包含共享专家，与Qwen3MoE等无共享专家的设计不同
OLMo2	注重透明度，训练数据和代码公开；采用Post-Norm归一化层位置设计和QK-Norm；使用传统MHA	1.Post-Norm归一化层位置：置于注意力和前馈模块后，结合残差连接助力训练稳定;2.QK-Norm：在注意力机制中对查询和键应用RMSNorm，增强训练稳定性	1.坚持使用传统MHA，未采用MLA或GQA;2.归一化层设计独特，与GPT的Pre-Norm、原始Transformer的Post-Norm不同
Gemma3	270亿参数模型性能与效率平衡佳；使用滑动窗口注意力；归一化层同时采用Pre-Norm和Post-Norm	1.滑动窗口注意力：调整全局与局部注意力比例为5:1，窗口大小减至1024，大幅减少KV缓存内存；2.归一化层同时采用Pre-Norm和Post-Norm，兼顾两者优势	1.采用滑动窗口注意力优化效率，与DeepSeek的MoE、Mistral的常规GQA不同；2.归一化层设计结合Pre和Post，独具特色
MistralSmall3.1	240亿参数，性能优于Gemma327B且速度更快；使用常规分组查询注意力	1.优化分词器、缩减KV缓存和层数以降低推理延迟；2.放弃滑动窗口注意力，改用常规GQA以利用更优代码优化（如FlashAttention）	1.专注降低推理延迟，性能优于同规模部分模型；2.放弃滑动窗口注意力，与Gemma3不同
Llama4	4000亿参数，采用MoE架构；交替使用MoE和密集模块；使用GQA	1.MoE架构中专家数量少但规模大，每次激活2个专家；2.交替使用MoE和密集模块，平衡性能与效率	1.MoE设计中专家规模大、数量少，与DeepSeek的多小专家不同；2.注意力机制用GQA，与DeepSeek的MLA不同
Qwen3	含密集（0.6B-32B）和MoE（30B、235B）版本；密集模型中小规模模型性能佳；MoE模型无共享专家	1.密集模型中0.6B为当前最小开源模型之一，性能优异；2.MoE模型取消共享专家，可能为节省计算和内存成本	1.同时提供密集和MoE版本，满足不同场景需求；2.MoE模型无共享专家，与DeepSeek的MoE设计不同
SmolLM3	30亿参数，性能良好；共享大量训练细节；采用NoPE理念	1.采用NoPE理念，每4层省略RoPE，利于长序列泛化；2.训练细节公开，类似OLMo，便于研究	1.规模介于1.7B和4B模型之间，提供中间选择；2.应用NoPE理念，与其他使用RoPE的模型不同
Kimi2	1万亿参数，性能媲美顶级专有模型；使用Muon优化器；基于DeepSeek-V3架构扩大规模	1.首次在超大规模模型（1万亿参数）中使用Muon优化器，训练损失曲线衰减良好；2.基于DeepSeek-V3架构，扩大规模，调整专家数量和注意力头数	1.参数规模达1万亿，为当前同代最大LLM之一；2.使用Muon优化器，与其他使用AdamW的模型不同
GLM4.5	3550亿总参数MoE架构，激活320亿参数；原生融合ARC三大能力（智能体、推理、编程）	1.深度优化的MoE路由：采用无损平衡路由和Sigmoid门控机制，确保专家负载均衡；2.混合推理模式：支持思考模式（慢速高精度）和非思考模式（高速实时）的智能切换；3.三阶段训练流程：预训练→中期训练→后训练，针对性强化不同能力	1.架构设计：采用深而非宽的设计（89层，隐藏维度5120），与DeepSeek-V3理念相似但参数配置不同；2.注意力机制：使用GQA（96头）+RoPE（基频1M），与DeepSeek的MLA、Gemma的滑动窗口注意力形成差异；3.训练策略：独特的智能体轨迹数据注入和迭代自蒸馏技术，实现ARC能力原生融合

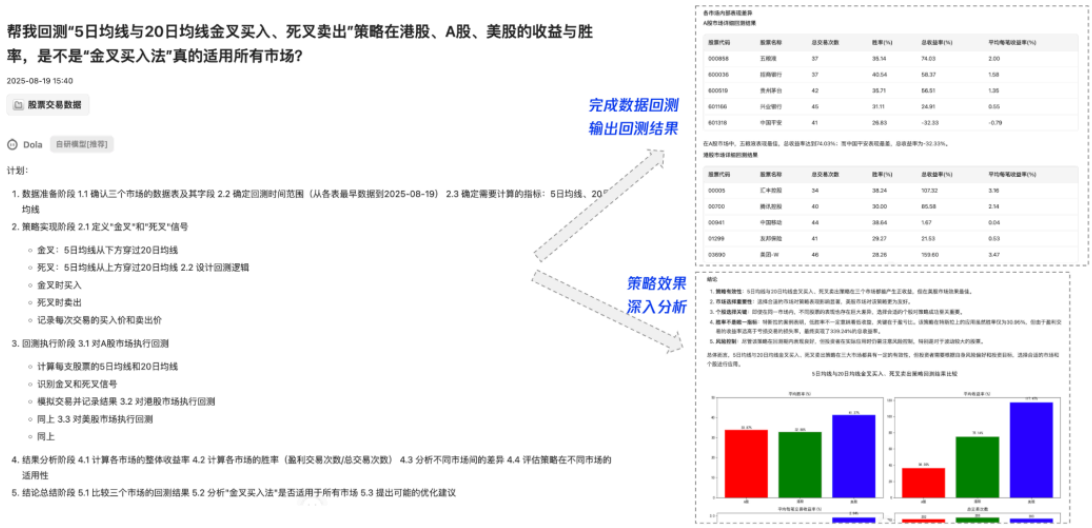
>>插个题外话：

随着大模型进化而进化的，还有腾讯PCG大数据平台部的**新一代数据分析AI助手——Dola**，以全新的面貌与大家相见啦～



Dola是一款基于AgenticAI能力开发的数据分析助手：用户只需要引入个人的数据表，就能得到一枚专属的AI分析师。它不仅能够完成日常的取数、跑数等基础任务，还能自主规划并执行复杂场景的数据分析，例如异动归因、画像对比分析、股票基金回测、房价预测等。Dola可以自行编写SQL、纠正SQL错误、执行查询、使用Python进行数据处理与可视化，并最终生成一份完整的分析报告。**全程无需编写一行代码，只需通过自然语言对话，你就能拥有一个全自动工作的"数据小黑工"。**

这里以1个股票回测的例子看看dola的效果：



可以看到dola在接收到“金叉买入法回测”这个问题之后，首先能自己生成一个计划，包括了需要进行的数据准备、策略实现、回测实现和结果分析等详细内容。在按照自己的计划执行完毕后，dola最终产出回测结果，完成可视化并进行深入分析总结，产出一份完整的回测报告。这里只是以股票回测这样一个比较复杂的案例场景展开，大家应该可以想象到，在工作场景中，Dola对于日常数据分析工作的提效程度是显而易见的。

回到主题，本期分享超长文预警，对大模型感兴趣的同学欢迎接着往下阅读：

一、大模型发展介绍

间隔分享deepseek系列过去了小半年，在这段时间开源大语言模型（LLM）领域迎来新一轮爆发式增长。在业务需求与技术探索的双重驱动下，模型能力的边界被不断突破，从参数规模、训练范式到应用性能均发生显著迭代。DeepSeek、Claude、Kimi2等系列模型的持续进化，标志着大模型技术已从“可用”迈向“高效可靠”，其背后隐藏的核心变化亟待系统梳理。

当前，主流开源模型的提升不仅体现在基准测试分数的跃升，更反映在三大维度：

1.**能力质变**：逻辑推理、长上下文理解、多模态交互等关键场景的突破性进展；

2.**效率革命**：稀疏化训练、动态架构等技术大幅降低推理成本；

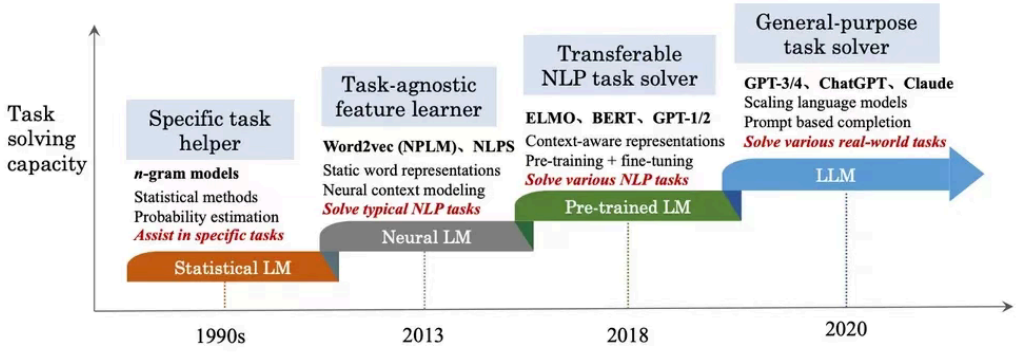
3.**生态重构**：垂直领域微调、智能体协作框架推动工业化落地。

本文旨在拆解近期大模型的技术演进脉络，为开发者与从业者提供一份快速认知地图——我们究竟该如何理解这场静默却深刻的能力变革？

（注：文章内容较多，如有疏漏，欢迎指出修正~）

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

1.语言模型的发展历程



语言模型的发展历程，可以划分为四个关键阶段，每个阶段在模型架构、训练范式和应用能力上都实现了质的突破。

1.1.统计语言模型阶段（1990年代）

作为NLP技术发展的起点，统计语言模型（StatisticalLM）采用n-gram统计方法和概率估计（如马尔可夫假设）进行词序列预测。这类模型主要服务于特定任务（如拼写检查、简单机器翻译），其技术特征表现为：

●基于人工设计的统计特征

●依赖局部上下文窗口（通常2-3个词）

●面临数据稀疏性和长距离依赖捕捉困难等核心挑战

1.2.神经语言模型阶段（2013年）

神经网络架构的引入催生了神经语言模型（NeuralLM）的革命。通过RNN、LSTM等网络结构学习词的分布式表示（词向量），该阶段实现了：

●自动特征学习能力，摆脱人工特征工程

●词向量的语义空间映射（如Word2Vec的"国王-男人+女人≈女王"）

●模型参数规模提升至百万量级

●仍需针对不同任务进行定制化训练

1.3.预训练语言模型阶段（2018年）

Transformer架构的提出推动NLP进入预训练时代。预训练语言模型（Pre-trainedLM）通过"预训练+微调"范式：

●采用自监督学习（如BERT的掩码语言建模）

●参数规模突破亿级（BERT-base:110M）

●实现上下文敏感表示（ELMo）和双向特征编码

●支持跨任务知识迁移（如GLUE基准测试）

●典型代表包括BERT、GPT-1/2等

1.4.大语言模型阶段（2020年至今）

千亿参数规模的大语言模型（LLM）标志着NLP进入新纪元。其核心突破在于：

●模型架构：基于稠密激活的Transformer变体

●训练范式：提示工程（PromptEngineering）实现零样本推理

●规模效应：遵循scalinglaws的性能提升规律

●涌现能力：包括思维链（Chain-of-Thought）推理等

●典型代表：GPT-3/4（175B参数）、PaLM（540B）

大模型真正走向大众视野是在2022年年末ChatGPT的推出。

2.开源文本大模型核心点速读

●**DeepSeekV3/R1**：采用**多头潜在注意力（MLA）**替代分组查询注意力（GQA），结合混合专家（MoE）架构，6710亿总参数仅激活370亿进行推理，提升效率与性能。

●**OLMo2**：注重透明度，采用**Post-Norm归一化层位置设计和QK-Norm（对查询和键应用RMSNorm）**，有助于训练稳定，仍使用传统多头注意力（MHA）。

●**Gemma3**：通过**滑动窗口注意力**减少KV缓存内存需求，调整全局与局部注意力比例为5:1，归一化层同时采用Pre-Norm和Post-Norm。

●**MistralSmall3.1**：240亿参数模型在多个基准测试中优于Gemma327B且速度更快，使用常规分组查询注意力以降低推理延迟。

●**Llama4**：采用MoE架构，与DeepSeek-V3类似但专家数量更少、规模更小，**交替使用MoE和密集模块**。

●**Qwen3**：包含密集和MoE版本，密集模型中0.6B参数模型体积小、性能佳；MoE模型无共享专家，适合不同场景。**2507更新了三个模型，发布了专注软件开发的Qwen3-Coder-480B-A35B-Instruct。**

●**SmolLM3**：30亿参数模型性能良好，每4层省略RoPE（采用**NoPE理念**），利于长序列泛化。

●**Kimi2**：1万亿参数，使用**Muon优化器**，基于**DeepSeek-V3架构扩大规模**，性能媲美顶级专有模型。

●**GLM-4.5**：采用**深度优化**的MoE架构（3550亿总参数仅激活320亿），原生融合智能体、推理与编程三大能力，通过三阶段训练和混合推理模式实现高性能与高效率的平衡。

九大模型对比表

模型名称	核心点	创新点	与其他模型的不同点
DeepSeekV3/R1	6710亿总参数，采用MoE架构，推理时仅激活370亿参数；使用MLA替代GQA	1.多头潜在注意力（MLA）：压缩键值张量至低维空间存储，提升性能且减少KV缓存内存；2.MoE架构含共享专家，始终活跃，助力学习通用模式与专业模式	1.注意力机制上，用MLA而非GQA，且MLA性能优于MHA和GQA2.MoE架构中包含共享专家，与Qwen3MoE等无共享专家的设计不同
OLMo2	注重透明度，训练数据和代码公开；采用Post-Norm归一化层位置设计和QK-Norm；使用传统MHA	1.Post-Norm归一化层位置：置于注意力和前馈模块后，结合残差连接助力训练稳定;2.QK-Norm：在注意力机制中对查询和键应用RMSNorm，增强训练稳定性	1.坚持使用传统MHA，未采用MLA或GQA;2.归一化层设计独特，与GPT的Pre-Norm、原始Transformer的Post-Norm不同
Gemma3	270亿参数模型性能与效率平衡佳；使用滑动窗口注意力；归一化层同时采用Pre-Norm和Post-Norm	1.滑动窗口注意力：调整全局与局部注意力比例为5:1，窗口大小减至1024，大幅减少KV缓存内存；2.归一化层同时采用Pre-Norm和Post-Norm，兼顾两者优势	1.采用滑动窗口注意力优化效率，与DeepSeek的MoE、Mistral的常规GQA不同；2.归一化层设计结合Pre和Post，独具特色
MistralSmall3.1	240亿参数，性能优于Gemma327B且速度更快；使用常规分组查询注意力	1.优化分词器、缩减KV缓存和层数以降低推理延迟；2.放弃滑动窗口注意力，改用常规GQA以利用更优代码优化（如FlashAttention）	1.专注降低推理延迟，性能优于同规模部分模型；2.放弃滑动窗口注意力，与Gemma3不同
Llama4	4000亿参数，采用MoE架构；交替使用MoE和密集模块；使用GQA	1.MoE架构中专家数量少但规模大，每次激活2个专家；2.交替使用MoE和密集模块，平衡性能与效率	1.MoE设计中专家规模大、数量少，与DeepSeek的多小专家不同；2.注意力机制用GQA，与DeepSeek的MLA不同
Qwen3	含密集（0.6B-32B）和MoE（30B、235B）版本；密集模型中小规模模型性能佳；MoE模型无共享专家	1.密集模型中0.6B为当前最小开源模型之一，性能优异；2.MoE模型取消共享专家，可能为节省计算和内存成本	1.同时提供密集和MoE版本，满足不同场景需求；2.MoE模型无共享专家，与DeepSeek的MoE设计不同
SmolLM3	30亿参数，性能良好；共享大量训练细节；采用NoPE理念	1.采用NoPE理念，每4层省略RoPE，利于长序列泛化；2.训练细节公开，类似OLMo，便于研究	1.规模介于1.7B和4B模型之间，提供中间选择；2.应用NoPE理念，与其他使用RoPE的模型不同
Kimi2	1万亿参数，性能媲美顶级专有模型；使用Muon优化器；基于DeepSeek-V3架构扩大规模	1.首次在超大规模模型（1万亿参数）中使用Muon优化器，训练损失曲线衰减良好；2.基于DeepSeek-V3架构，扩大规模，调整专家数量和注意力头数	1.参数规模达1万亿，为当前同代最大LLM之一；2.使用Muon优化器，与其他使用AdamW的模型不同
GLM4.5	3550亿总参数MoE架构，激活320亿参数；原生融合ARC三大能力（智能体、推理、编程）	1.深度优化的MoE路由：采用无损平衡路由和Sigmoid门控机制，确保专家负载均衡；2.混合推理模式：支持思考模式（慢速高精度）和非思考模式（高速实时）的智能切换；3.三阶段训练流程：预训练→中期训练→后训练，针对性强化不同能力	1.架构设计：采用深而非宽的设计（89层，隐藏维度5120），与DeepSeek-V3理念相似但参数配置不同；2.注意力机制：使用GQA（96头）+RoPE（基频1M），与DeepSeek的MLA、Gemma的滑动窗口注意力形成差异；3.训练策略：独特的智能体轨迹数据注入和迭代自蒸馏技术，实现ARC能力原生融合

3.闭源文本大模型新闻速读

因为闭源缘故，缺少相关的资料，这部分模型只能做一些新闻的总结。本文核心也将放在开源的模型架构介绍上。

●GPT5：路由架构。利用多个模型来自动路由和组合完成任务。

●Claude：Claude系列通过扩展上下文窗口（Sonnet4达100万token）、提升编程基准分数（Opus4.1在SWE-bench达74.5%）和强化安全策略，成为当前闭源模型中代码能力领先的AI。2025年9月29日发布，发布旗舰模型——ClaudeSonnet4.5，定位为“世界最佳编程模型”和构建复杂智能体的最强工具。ClaudeHaiku4.5是后发的轻量级模型，定位为高性价比、低延迟的实时任务解决方案。

二、开源大模型体系结构比较——从DeepSeek-V3到KimiK2、GLM4.6：现代LLM架构设计（2025年）

——从SebastianRaschka、大模型综述、模型技术报告等文章中整理

从ChatGPT的火热至今已有两个半的年头，目前的大模型与2019年的GPT-2架构仍有非常多的相似之处。有一些革新，例如：位置嵌入已从绝对位置演变为旋转（RoPE）位置，多头注意力在很大程度上已被分组查询注意力（GQA）所取代，更高效的SwiGLU已替代了GELU等激活函数，MOE架构和强化学习均成为分支路径。

疑问：大模型架构是否有真正突破性的变化？

因材料不翔实，大模型架构之间的比较充满挑战，下文大部分内容出自SebastianRaschka教授的文章，小部分从其他论文等整理得到。因资料受限，介绍2025年开源的头部模型架构为主，基准性能和训练算法则短期不在本文的主要关注点上。

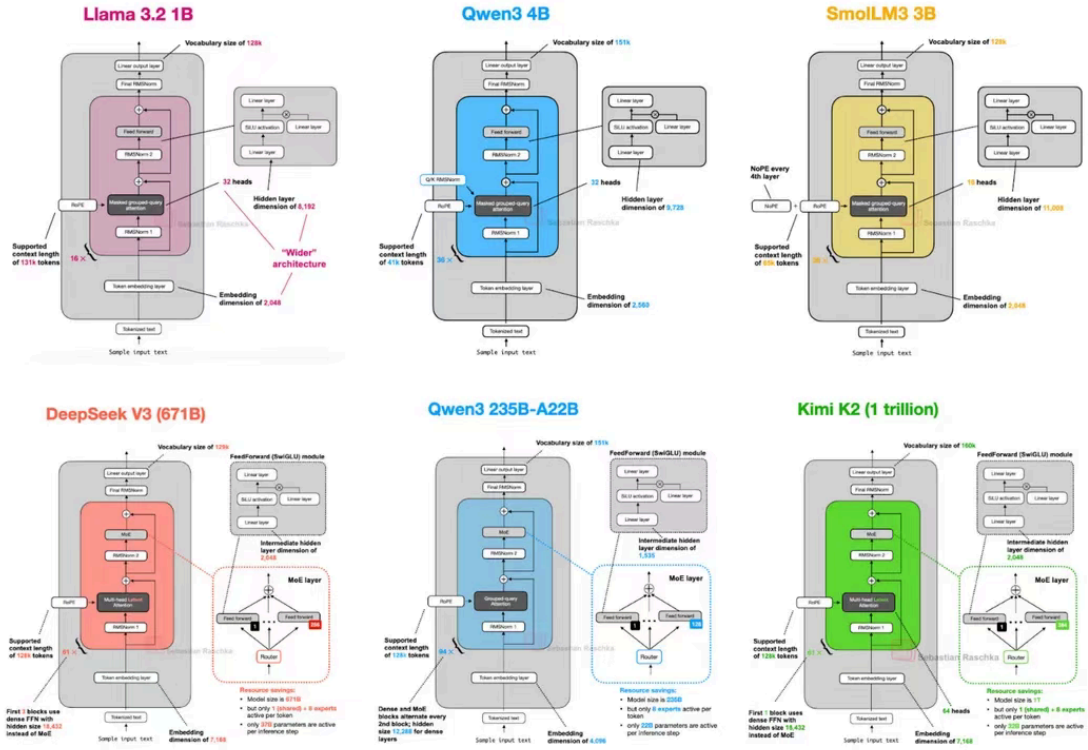


Figure 1: A subset of the architectures covered in this article.

1. DeepSeekV3/R1

| Deepseek系列笔者做过之前一些分享汇总，详情可见文章：【技术报告解读】以小白方式PPT图解汇总DeepSeek系列，共同领略DeepSeek-R1的前世今生

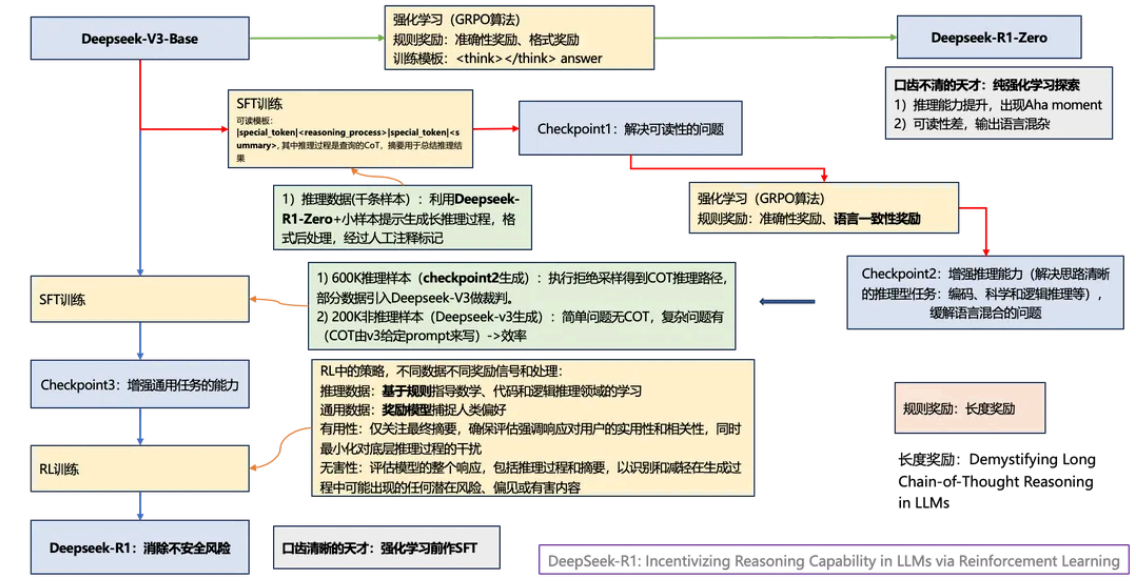
DeepSeek系列核心技术发展汇总

版本	核心创新	其他	发布时间	论文
DeepSeek LLM	基本是LLaMA架构。 1）提出超参数的扩展定律 2）使用多步学习率调度器替换余弦学习率调度器	模型架构基于LLaMA的设计，采用Pre-Norm结构和RMSNorm函数，使用SwiGLU作为前馈网络的激活函数。宏设计上，DeepSeek LLM 7B有30层，而67B有95层，以优化训练和推理。67B模型使用Grouped-Query Attention（GQA）以优化推理成本。	2024 年 1 月	DeepSeek LLM Scaling Open-Source Language Models with Longtermism
DeepSeekMoE	1）细粒度专家细分：在维持参数总量恒定的前提下，将专家模块进行更精细的划分 2）共享专家隔离：将部分专家设定为共享专家，这些专家在模型运行过程中始终保持激活状态，专门用于捕获和整合通用知识。 3）负载均衡考虑（专家和设备级loss）	在总参数和激活参数数量相同的情况下，DeepSeekMoE 表现出优于 GShard 的压倒性优势，实现了与 **GShard×1.5** 相当的性能	2024 年 1 月	DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Model
DeepSeek-Math	1）DeepSeekMath语料库构建 2）**组相对策略优化(GRPO)算法**：改进 PPO 算法，通过组分数估计基线简化价值函数	1）发现代码训练能提升模型数学推理能力 2）**统一范式分析多种训练方法**：提供统一范式理解 SFT、RFT、DPO、PPO 和 GRPO 等训练方法	2024 年 2 月	DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models
DeepSeek-V2	**多头潜在注意力（MLA）**：通过低秩键值联合压缩，将键值对压缩为潜在向量，大幅减少推理时的 KV 缓存，同时采用解耦旋转位置嵌入（RoPE）策略解决与低秩 KV 压缩的兼容性问题，在保证性能的同时提升推理效率。	MLA+DeepSeekMOE+GRPO，大幅吹起开创号角！ 训练成本比 DeepSeek 67B 节省 42.5%，推理时通过 FP8 量化和 KV 缓存量化，减少 KV 缓存，提升生成吞吐量，是 DeepSeek 67B 的 5.76 倍。	2024 年 5 月	DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model
DeepSeek V3	**极致成本：1/10 llama-70B的训练成本，1/30 openai的推理价格** 1）**使用多token预测机制（Multi-Token Prediction, MTP）**：创新性地采用了 MTP 目标，将预测范围**扩展到每个位置的多个后续 token。** 2）**训练优化创新**：FP8混合精度训练框架+高效训练框架设计（DualPipe 算法+高效的跨节点全对全通信内核）	知识蒸馏：从 DeepSeekR1 系列模型中提取推理能力，并融入到 DeepSeek-V3 中，提升推理能力。 MLA+DeepSeekMOE+GRPO+无辅助损失负载平衡+MTP等，用极低资源创造极高效能。	2024 年 12 月	DeepSeek-V3 Technical Report
DeepSeek-R1	1）**开创纯强化学习训练**：使用GRPO进行低成本训练，采用基于规则的奖励系统，包括准确性奖励和格式奖励，引导模型提升推理能力 2）RL前增加少样本SFT效果更加。	在RL激发推理能力赛道中开源的顶流，使用更少资源超越了01性能。	2025 年 1 月	DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeekR1在2025年1月发布时产生了巨大影响。DeepSeekR1是构建在2024年12月推出的DeepSeekV3架构之上的推理模型。

如果对DeepSeekR1的训练有感兴趣，也可以见上述DeepSeek系列文章。

核心问题解释：R1的推理能力怎么通过训练诞生的？图解DeepSeek-R1训练和数据生成流程



下文重点介绍DeepSeekV3中引入的两项关键架构技术，这些技术提高了其计算效率，使其有别于许多其他大型语言模型（LLM）：

●多头潜在注意力（MLA）

●专家混合（MoE）

1.1多头潜在注意力（MLA）

在讨论多头潜在注意力（MLA）之前，让我们简要回顾一些背景信息，以了解其使用的原因。为此，让我们从分组查询注意力（GQA）开始，近年来它已成为替代多头注意力（MHA）的计算和参数效率更高的标准选择。因此，以下是GQA的简要概述。与MHA不同，在MHA中

每个头部都有自己的键和值集，为了减少内存使用，GQA将多个头部组合在一起共享相同的键和值投影。

例如，如下图2进一步所示，如果有2个键值组和4个注意力头部，那么头部1和2可能共享一组键和值，而头部3和4共享另一组。这减少了键和值计算的总数，从而降低了内存使用量（参见多项研究）。

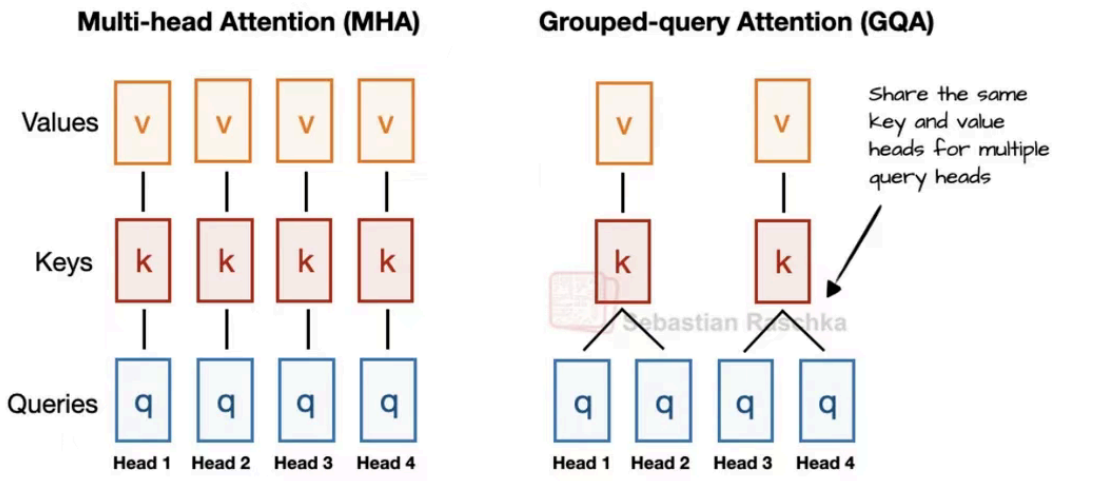


Figure 2: A comparison between MHA and GQA. Here, the group size is 2, where a key and value pair is shared among 2 queries.

所以，GQA的核心思想是通过在多个查询头之间共享关键值头来减少关键值头的数量。这（1）降低了模型的参数数量，（2）由于需要从键值缓存中存储和检索的关键值更少，因此在推理过程中减少了键值张量的内存带宽使用。

（如果您好奇GQA在代码中的实现，请参阅我的GPT-2到Llama3转换指南，其中包含一个没有键值缓存的版本，以及我在此处的键值缓存变体。）虽然GQA主要是针对多头自注意力机制（MHA）计算效率问题的一种解决方案，但消融研究（如原始GQA论文和Llama2论文中的研究）表明，在大型语言模型（LLM）建模性能方面，其表现与标准MHA相当。

现在，多头潜在注意力（MLA）提供了一种不同的内存节省策略，它也与键值缓存特别搭配。MLA不是像GQA那样共享关键值头，而是先将键值张量压缩到低维空间，然后再将其存储在键值缓存中。在推理时，这些被压缩的张量会被投影回其原始大小后再使用，如下图3所示。这增加了一个额外的矩阵乘法，但减少了内存使用。

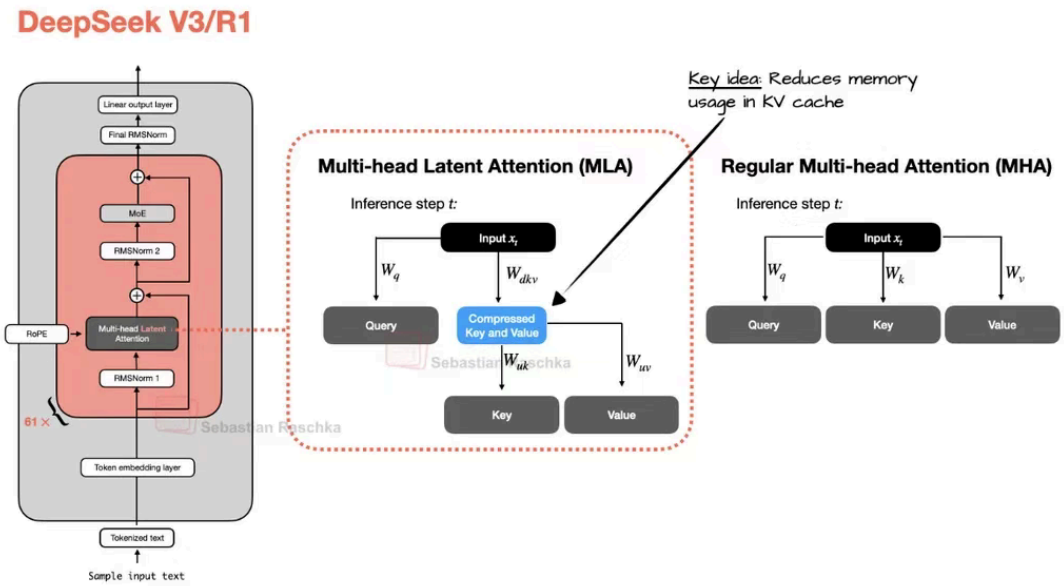
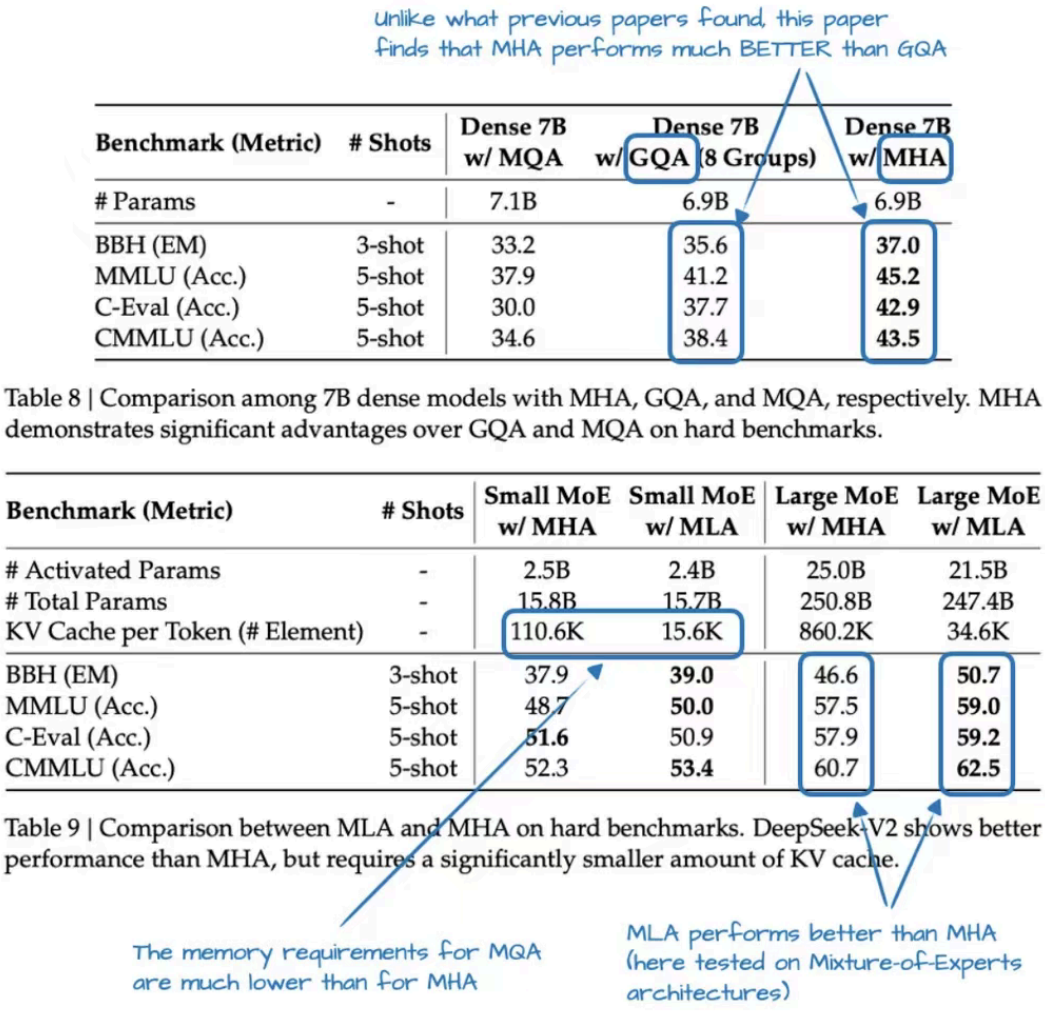


Figure 3: Comparison between MLA (used in DeepSeek V3 and R1) and regular MHA.

（作为补充说明，查询在训练过程中也被压缩了，但在推理过程中则没有。）

顺便说一下，MLA在DeepSeekV3中并不是新东西，因为其DeepSeek-V2的前身也曾使用过（甚至引入）它。此外，V2论文中还包含了一些有趣的消融研究，这些研究可能解释了为什么DeepSeek团队选择MLA而不是GQA（见下面的图4）。



Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	**37.0**
MMLU (Acc.)	5-shot	37.9	41.2	**45.2**
C-Eval (Acc.)	5-shot	30.0	37.7	**42.9**
CMMLU (Acc.)	5-shot	34.6	38.4	**43.5**

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	**39.0**	46.6	**50.7**
MMLU (Acc.)	5-shot	48.7	**50.0**	57.5	**59.0**
C-Eval (Acc.)	5-shot	**51.6**	50.9	57.9	**59.2**
CMMLU (Acc.)	5-shot	52.3	**53.4**	60.7	**62.5**

Table 9 | Comparison between MLA and MHA on hard benchmarks. DeepSeek-V2 shows better performance than MHA, but requires a significantly smaller amount of KV cache.

Figure 4: Annotated tables from the DeepSeek-V2 paper, https://arxiv.org/abs/2405.04434

表8|分别比较了7B密集模型与MHA、GQA和MQA的情况。在困难基准测试中，MHA相较于GQA和MQA表现出显著的优势。表9|在硬性基准测试中MLA与MHA的比较。DeepSeek-V2表现出比MHA更好的性能，但需要显著更小的键值缓存量。

如上图4所示，GQA的表现似乎不如MHA，而MLA提供了比MHA更好的建模性能，这很可能是DeepSeek团队选择MLA而非GQA的原因。（同样有趣的是，我们也很想看到MLA与GQA在“每个标记的KV缓存”节省方面的对比！）

在我们继续讨论下一个架构组件之前，总结一下这一部分，MLA是一种巧妙的技巧，可以在减少KV缓存内存使用的同时，甚至在建模性能方面略微超越MHA。

1.2专家混合（MoE）

在DeepSeek中值得重点介绍的其他主要架构组件是其使用专家混合（MoE）层。虽然DeepSeek并未发明MoE，但今年它又重新流行起来，我们稍后将要介绍的许多架构也采用了MoE。

您可能已经熟悉MoE，但快速回顾一下可能会有所帮助。MoE的核心思想是用多个专家层替换变换器块中的每个前馈模块，而这些专家层本身也是前馈模块。这意味着我们用多个前馈模

块替换了单个前馈模块，如下图5所示。

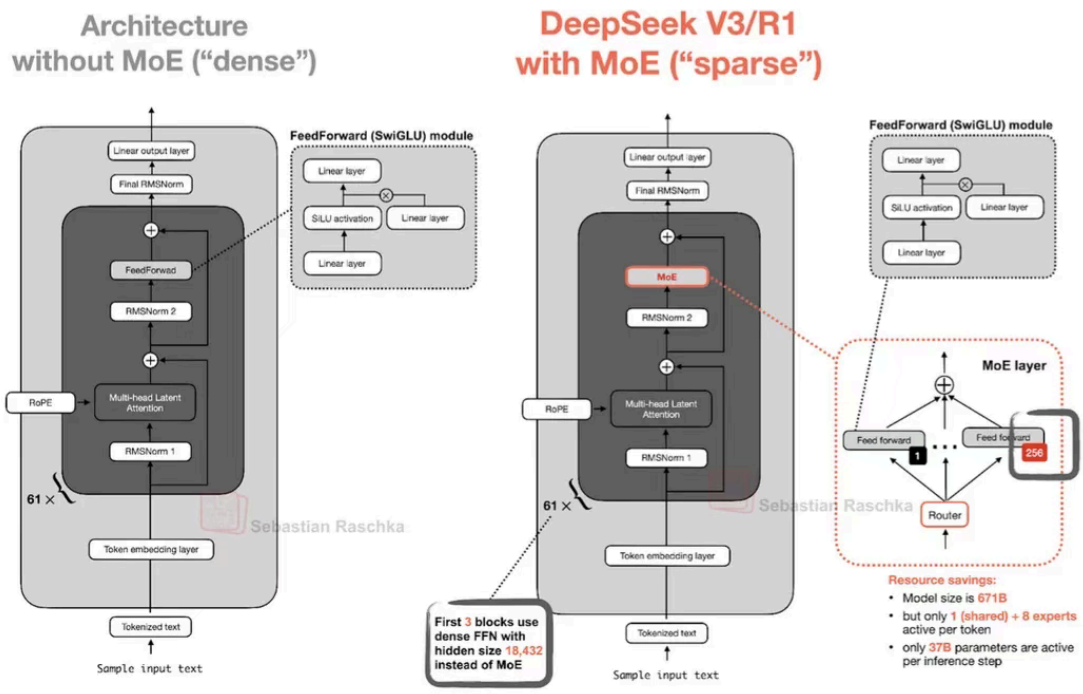


Figure 5: An illustration of the Mixture-of-Experts (MoE) module in DeepSeek V3/R1 (right) compared to an LLM with a standard FeedForward block (left).

MoE通常设置在attention层之后，在变压器块内部的前馈块（如上图中的深灰色块所示）通常包含大量模型的总参数。（请注意，在大型语言模型中，变压器块会重复多次；以DeepSeek-V3为例，共重复了61次。）

因此，用多个前馈块替换单个前馈块（如在MoE设置中所做的）会大幅增加模型的总参数数量。然而，关键技巧是我们并不对每个标记使用（“激活”）所有专家。相反，路由器每次只为每个标记选择一小部分专家（为了节省时间，或者更确切地说是文章篇幅，我将在另一个时间详细讨论路由器。）

因为一次只有少数专家处于活跃状态，MoE模块通常被称为稀疏的，与总是使用完整参数集的密集模块形成对比。然而，通过MoE的大量总参数增加了大型语言模型的容量，这意味着它在训练过程中可以吸收更多知识。尽管如此，稀疏性保持了推理的效率，因为我们不会同时使用所有参数。

例如，DeepSeek-V3的每个MoE模块有256个专家，总共有6710亿个参数。然而在推理过程中，每次只有9个专家处于活跃状态（1个共享专家加上路由器选择的8个）。这意味着每次推理步骤只使用了370亿个参数，而不是全部的6710亿个。

DeepSeek-V3的MoE设计的一个显著特点是使用了一个共享专家。这是一个对每个标记始终活跃的专家。这个想法并不新颖，已在DeepSeek2024MoE和2022DeepSpeedMoE论文中介绍过。

早期MoE：拥有更大但更少的专家，并且只激活少量专家（这里：2个）细粒度MoE使用更多但更小的专家，并激活更多专家（这里4个）具有共享专家的MoE也使用许多小专家，但增加了一个始终处于激活状态的共享专家。

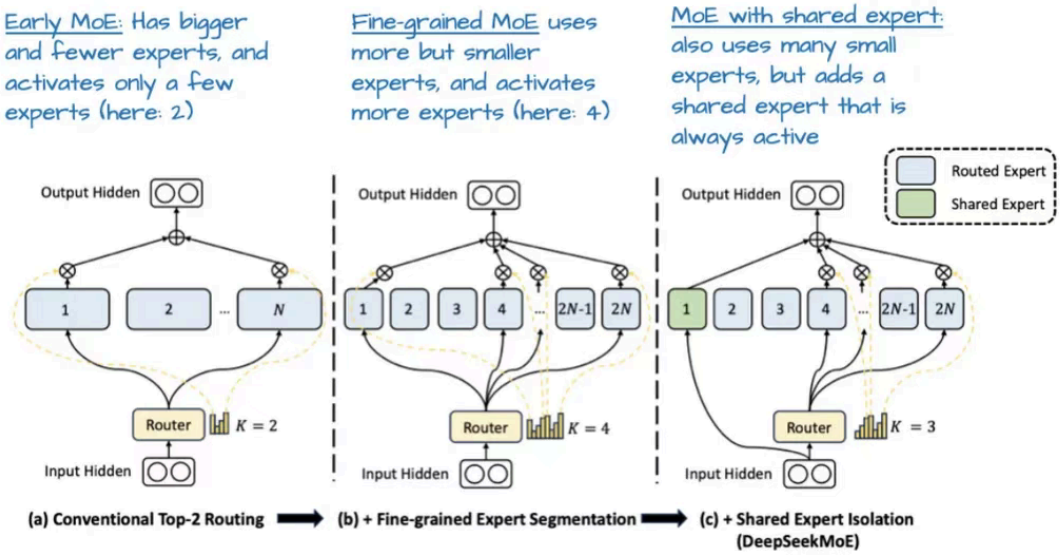


Figure 6: An annotated figure from "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models", https://arxiv.org/abs/2401.06066

在DeepSpeedMoE论文中首次指出了拥有共享专家的益处，他们发现与没有共享专家相比，这提高了整体建模性能。这很可能是因为常见或重复的模式不必由多个个体专家学习，这为他们学习更专业模式留下了更多空间。

1.3.DeepSeek摘要

总结来说，DeepSeek-V3是一个庞大的6710亿参数模型，在发布时，其性能超越了其他开源权重模型，包括405B参数的Llama3。尽管它更大，但由于采用了专家混合（MoE）架构，它在推理时间上更为高效，该架构每个标记仅激活一小部分（仅370亿）参数。

另一个关键区别特征是DeepSeek-V3使用了多头潜在注意力（MLA），而不是分组查询注意力（GQA）。MLA和GQA都是标准多头注意力（MHA）的高效推理替代方案，尤其是在使用键值缓存时。虽然MLA实现起来更复杂，但DeepSeek-V2论文中的一项研究表明，它的建模性能优于GQA。

1.4.NSA（Native Sparse Attention）

R1训练的时候留下了一个后续优化点——长上下文的处理：

●**长文本建模需求与挑战**：长文本建模对下一代大语言模型至关重要，但标准注意力机制计算复杂度高，处理长序列时成为关键延迟瓶颈，稀疏注意力机制为提高效率提供方向。

●**现有稀疏注意力方法的局限**：许多稀疏注意力方法在实际部署中无法实现理论上的加速，且大多仅关注推理阶段，缺乏训练阶段的有效支持。

后续DeepSeek发布了NSA的核心技术论文：Native Sparse Attention: Hardware-Aligned and NativelyTrainable Sparse Attention，专为超长文本处理打造的“注意力加速器”！传统模型处理长文本像“死记硬背”，算到冒烟还卡顿。NSA却能**智能筛选关键信息**，既省算力又保智商。

NSA引入了两个核心创新以对应于上述关键需求：

1.**硬件对齐的算法设计**：NSA通过优化块状稀疏注意力机制，充分利用现代硬件（如GPU的TensorCore）的计算能力，确保计算与内存访问的平衡，从而在推理和训练阶段实现显著的加速。

2.**可训练的稀疏注意力机制**：NSA支持端到端的训练，能够在减少预训练计算量的同时，保持模型的性能。这使得NSA不仅适用于推理阶段，还能在训练阶段大幅提升效率。

NSA的核心组成：

✅ **【Token Compression-压缩块（最上）】动态分层压缩**：把长文本“分块提炼”，留下精华，丢掉冗余。将长序列划分为块，通过MLP压缩为粗粒度表示（如32块→1块），保留全局语义。

✅ **【Token Selection-选择块（中间）】精准捞重点**：像学霸划重点，动态锁定关键段落，一个不漏！基于压缩结果动态筛选重要性最高的块，保留细粒度关键信息。

✅ **【Sliding Window-滑动窗口块（最下）】滑动窗口补细节**：局部上下文也不放过，全面理解无死角！**强制保留局部邻近tokens**，防止模型被局部模式主导。

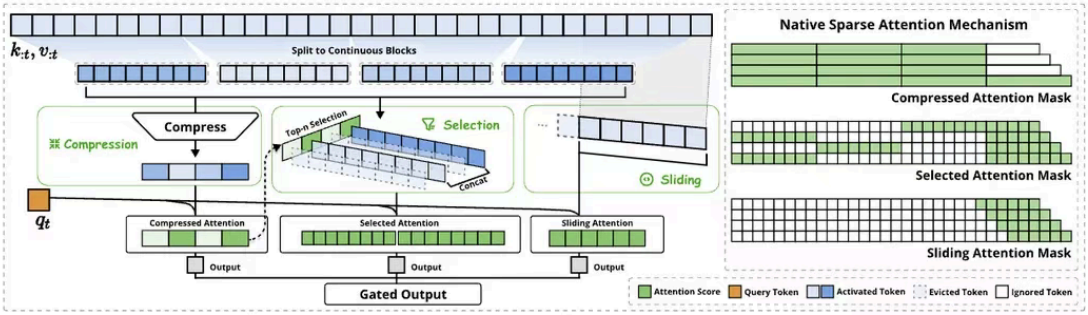


Figure 2 | Overview of NSA's architecture. Left: The framework processes input sequences through three parallel attention branches: For a given query, preceding keys and values are processed into compressed attention for coarse-grained patterns, selected attention for important token blocks, and sliding attention for local context. Right: Visualization of different attention patterns produced by each branch. Green areas indicate regions where attention scores need to be computed, while white areas represent regions that can be skipped.

1.5.DeepSeek-V3.2-Exp

DeepSeek-V3.2-Exp，通过引入DeepSeek稀疏注意力（DSA）来解决长上下文场景下的效率问题，目前DeepSeek新模型调用成本更低。DSA原型主要由两个组件组成：闪电索引器和细粒度令牌选择机制。

1）闪电索引器：该组件计算查询令牌和前一个令牌之间的索引分数，确定哪些令牌被查询令牌选中。索引分数的计算公式如下：

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \, cdot ReLU\left(q_{t,j}^I \, cdot k_s^I\right),$$

其中，H^I表示索引器头的数量；$q_{t,j}^I \in \mathbb{R}^{\mathbb{d}}$和$w_{t,j}^I \in \mathbb{R}$来自查询token$h_t$；$k_s^I \in \mathbb{R}^{d^I}$来自前面的token$h_s$。ReLU作为激活函数以提高计算效率。

2）**细粒度令牌选择机制**：基于索引分数，细粒度令牌选择机制仅检索前k个最高索引分数对应的键值对。然后，通过在查询令牌和稀疏选择的键值对之间应用注意力机制来计算注意力输出。

DeepSeek-V3.2-Exp模型训练过程：

DeepSeek-V3.2-Exp的训练过程基于DeepSeek-V3.1-Terminus，通过持续预训练和后训练完成，具体如下：

1）持续预训练

●**密集热身阶段**：使用短热身阶段初始化轻量索引器，保持密集注意力，冻结除索引器外的所有模型参数。聚合主要注意力分数生成目标分布，以KL散度损失为训练目标，用10的学习率训练1000步，处理21亿个token。

●**稀疏训练阶段**：引入细粒度token选择机制，优化所有模型参数。仅考虑选定token集，使索引器输出与主要注意力分布一致。索引器与主要模型分别按对应损失优化，学习率为7.3×10^-6，每个查询token选2048个键值token，训练15000步，处理9437亿个token。

2）后训练

●**专家蒸馏**：为各任务开发专家模型，从预训练基础检查点微调，经大规模RL计算训练。用不同模型生成训练数据，专家模型生成特定领域数据，蒸馏数据训练的模型后续经RL训练可提升性能。

●**混合RL训练**：采用组相对策略优化（GRPO）算法，将推理、代理和人类对齐训练合并为一个RL阶段。推理和代理任务采用基于规则的奖励等，一般任务采用生成式奖励模型，平衡长度与准确性、语言一致性与准确性的权衡。


腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

2. OLMo2

由非营利组织Allen人工智能研究所开发的OLMo系列模型因其训练数据和代码的透明度以及相对详细的技术报告而值得注意。

虽然你可能不会在任何基准测试或排行榜的顶部找到OLMo模型，但它们相当干净，更重要的是，由于其透明度，它们是开发大型语言模型（LLMs）的优秀蓝图。

而OLMo模型因其透明性而受到欢迎，但它们的表现也不差。实际上，在1月份（在Llama4、Gemma3和Qwen3之前）发布时，OLMo2模型在计算与性能的帕累托前沿上，如下图7所示。

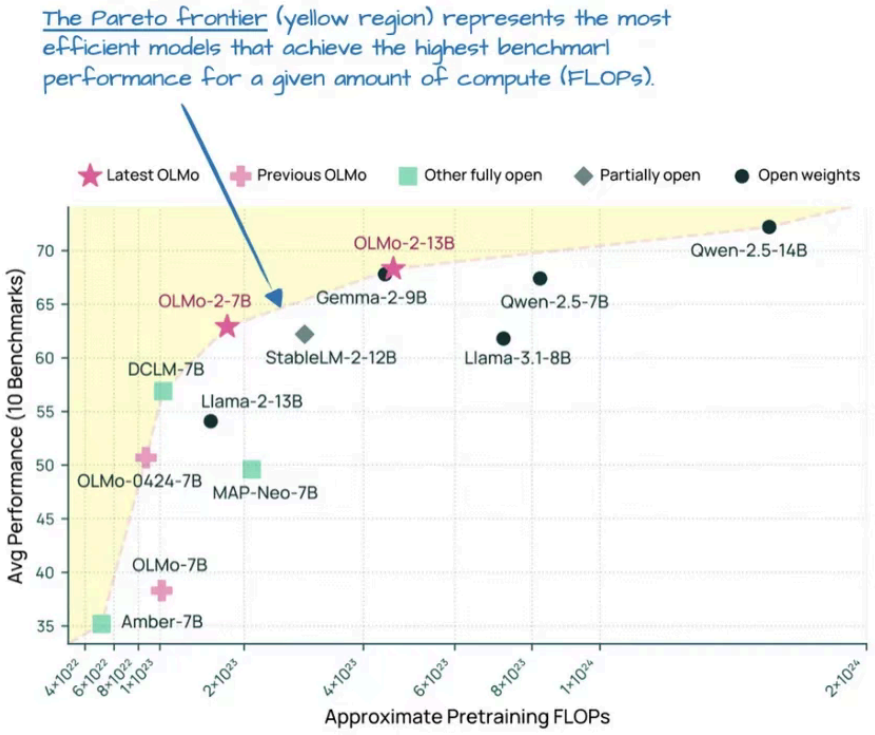


Figure 7: Modeling benchmark performance (higher is better) vs pre-training cost (FLOPs; lower is better) for different LLMs. This is an annotated figure from the OLMo 2 paper, https://arxiv.org/abs/2501.00656

正如本文前面提到的，我旨在仅关注LLM架构细节（而非训练或数据），以保持其长度在可管理范围内。那么，在OLMo2中有哪些有趣的架构设计选择呢？主要涉及规范化处理：RMSNorm层的放置以及添加QK规范化，我将在下文讨论。

另一件值得一提的事情是，OLMo2仍然使用传统的多头注意力（MHA），而不是MLA或GQA。

2.1规范化层放置

总体而言，OLMo2在很大程度上遵循了原始GPT模型的架构，与其他当代大型语言模型相似。然而，也有一些值得注意的差异。让我们从规范化层开始。与Llama、Gemma和大多数其他大型语言模型一样，OLMo2也从LayerNorm切换到RMSNorm。

但由于RMSNorm已经过时（它基本上是LayerNorm的简化版本，具有较少的可训练参数），我将跳过RMSNorm与LayerNorm的讨论。（好奇的读者可以在**LLMs-from-scratch/ch05/07_gpt_to_llama/converting-gpt-to-llama2.ipynbatmain·rasbt/LLMs-from-scratch·GitHub**中找到RMSNorm代码实现。）RMSNorm函数：归一化技术，旨在替代传统的LayerNormalization（层归一化）。相关论文：RMSNorm：Root Mean Square Layer Normalization。

归一化技术（如BatchNorm、LayerNorm等）被广泛用于加速训练并提高模型性能。LayerNorm通过对每个样本的特征进行归一化，减少了内部协变量偏移问题。然而，LayerNorm的计算依赖于均值和方差，这可能在某些情况下引入额外的计算开销。RMSNorm通过简化归一化过程，去除了对均值的依赖，仅使用均方根（Root Mean Square,RMS）进行归一化，从而降低了计算复杂度。

然而，讨论RMSNorm层的放置是值得的。原始的Transformer论文中提出的将两个规范化层分别放置在Transformer模块中的注意力模块和前馈模块之后。这也被称作后规范化（Post-LN）或后规范化（Post-Norm）。

RMSNorm 的核心思想是**仅使用特征的均方根值进行归一化**，而不像 LayerNorm 那样同时使用均值和方差。具体来说，RMSNorm 的计算步骤如下：

- 对于一个输入向量 $x = (x_1, x_2, \ldots, x_n)$，计算其均方根值：

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

- 对输入向量进行归一化：

$$\hat{x}_i = \frac{x_i}{\mathrm{RMS}(x)}$$

- 对归一化后的结果进行缩放和平移：

$$y_i = \gamma_i \cdot \hat{x}_i + \beta_i$$

其中，γ_i 和 β_i 是可学习的参数，用于恢复模型的表达能力。

特性	RMSNorm	LayerNorm
归一化方式	仅使用均方根（RMS）	使用均值和方差
计算复杂度	更低（无需计算均值）	较高
可学习参数	缩放参数 γ 和平移参数 β	同 RMSNorm
适用场景	高维数据、大规模模型	通用场景

GPT以及其后的大多数大型语言模型将规范化层放置在注意力模块和前馈模块之前，这被称作前规范化（Pre-LN）或前规范化（Pre-Norm）。下图展示了后规范化与前规范化的比较。

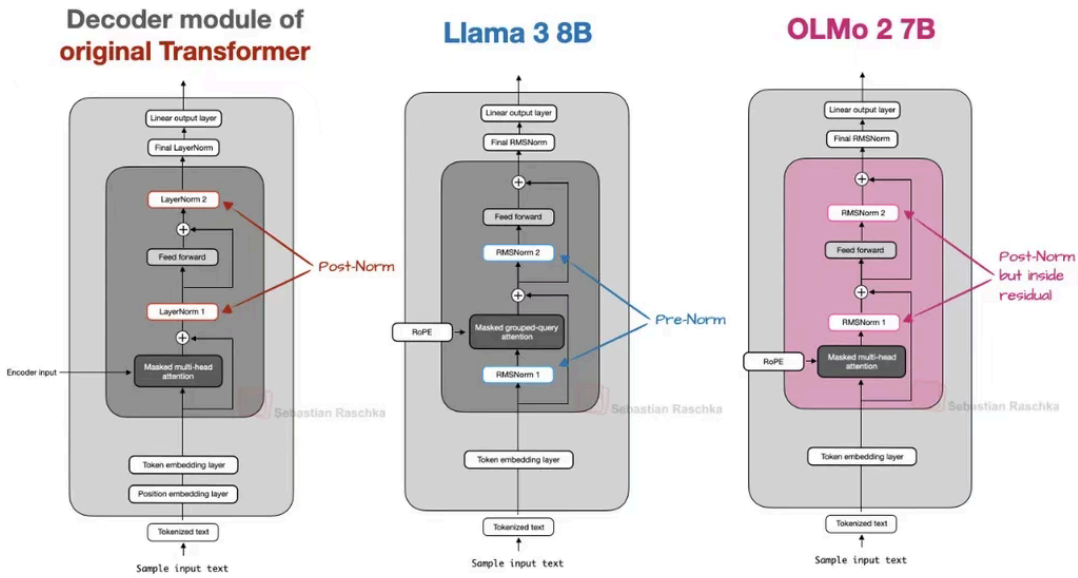


Figure 8: A comparison of Post-Norm, Pre-Norm, and OLMo 2's flavor of Post-Norm.

在[2002.04745]OnLayerNormalizationintheTransformerArchitecture展示了预层归一化（Pre-LN）能够在初始化时产生更稳定的梯度。此外，研究人员提到，即使没有经过仔细的学习率热身，预层归一化也能很好地工作，而学习率热身对于后层归一化（Post-LN）来说通常是一个关键工具。

我提到这一点的原因是，OLMo2采用了一种后层归一化的形式（但使用了RMS范数而非层范数，因此我称之为后范数）。在OLMo2中，不是将归一化层放在注意力机制和前馈层之前，而是将它们放在后面，如图所示。然而，请注意，与原始的Transformer架构相比，归一化层仍在残差层（跳跃连接）内部。那么，他们为什么要移动归一化层的位置呢？原因在于这有助于提高训练稳定性，如下图所示。

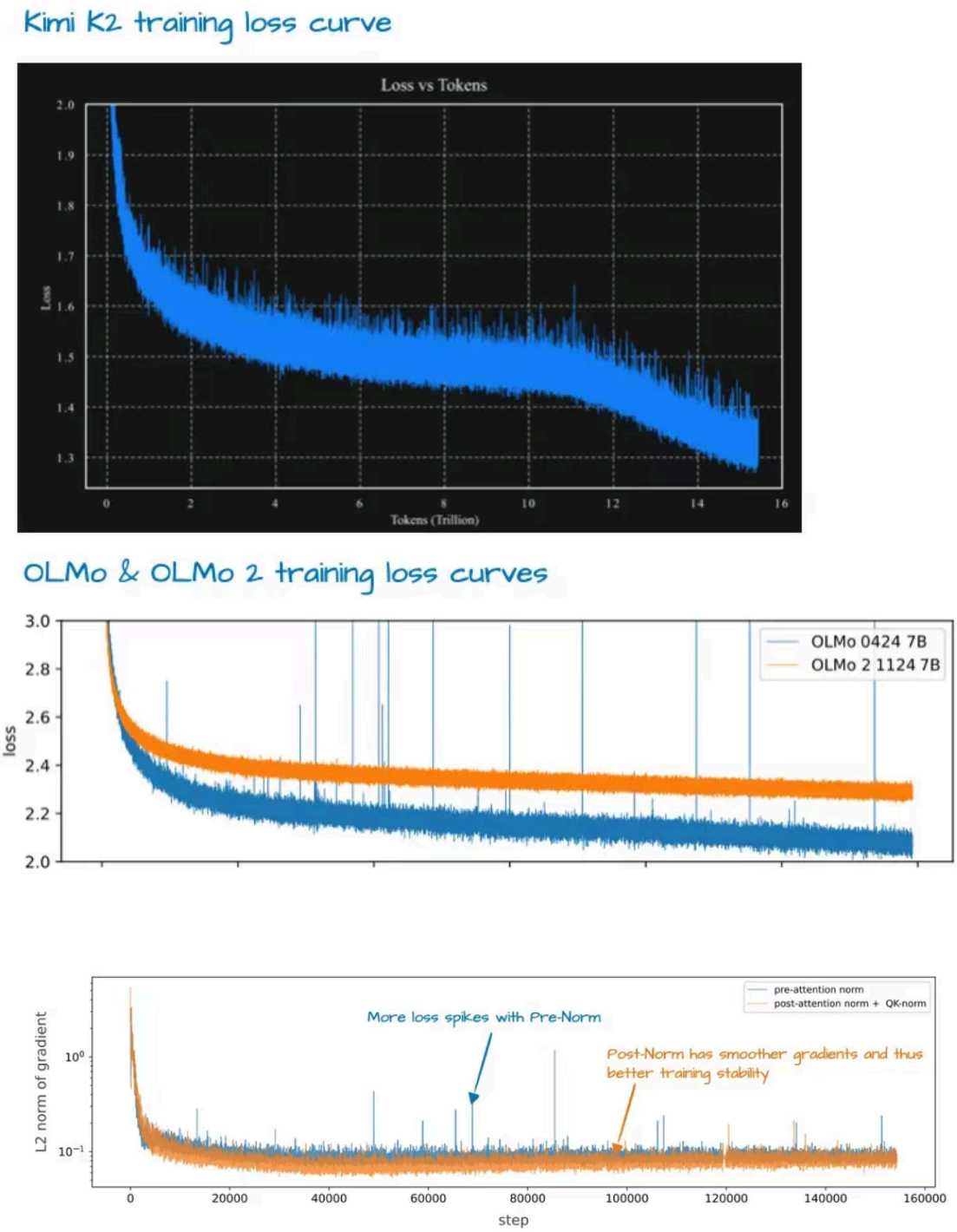


Figure 9: A plot showing the training stability for Pre-Norm (like in GPT-2, Llama 3, and many others) versus OLMo 2's flavor of Post-Norm.

不幸的是，这张图表显示了重新排序的结果以及QK-范数，后者是一个独立的概念。因此，很难判断归一化层重新排序自身贡献了多少。

2.2QK-范数

由于前一节已经提到了QK-范数，而且我们稍后讨论的其他大型语言模型（LLM），如Gemma2和Gemma3，也使用QK-范数，让我们简要讨论一下这是什么。

QK-范数本质上又是一个RMS范数层。它位于多头注意力（MHA）模块内部，并在应用RoPE之前应用于查询（q）和键（k）。为了说明这一点，下面是我为从头开始实现的Qwen3所写的

一个分组查询注意力（GQA）层的摘录（GQA中的QK范数应用类似于OLMo中的MHA）：

LLMs-from-scratch/ch05/11_qwen3atmain·rasbt/LLMs-from-scratch·GitHub

```
classGroupedQueryAttention(nn.Module):
def__init__(
self,d_in,num_heads,num_kv_groups,
head_dim=None,qk_norm=False,dtype=None
):
#...

ifqk_norm:
self.q_norm=RMSNorm(head_dim,eps=1e-6)
self.k_norm=RMSNorm(head_dim,eps=1e-6)
else:
self.q_norm=self.k_norm=None

defforward(self,x,mask,cos,sin):
b,num_tokens,_=x.shape

#Applyprojections
queries=self.W_query(x)
keys=self.W_key(x)
values=self.W_value(x)

#...

#Optionalnormalization
ifself.q_norm:
queries=self.q_norm(queries)
ifself.k_norm:
keys=self.k_norm(keys)

#ApplyRoPE
queries=apply_rope(queries,cos,sin)
keys=apply_rope(keys,cos,sin)

#ExpandKandVtomatchnumberofheads
keys=keys.repeat_interleave(self.group_size,dim=1)
values=values.repeat_interleave(self.group_size,dim=1)

#Attention
attn_scores=queries@keys.transpose(2,3)
#...
```

如前所述，结合Post-Norm，QK-Norm稳定了训练。请注意，QK-Norm并非由OLMo2发明，而是可以追溯到2023年的缩放视觉变换器论文

（[2302.05442]ScalingVisionTransformerstо22BillionParameters）。

2.3OLMo2总结

简而言之，值得注意的OLMo2架构设计决策主要是RMSNorm的位置安排：在注意力模块和前馈模块之后（一种Post-Norm变体），而不是之前使用RMSNorm，以及在注意力机制内部对查询和键添加RMSNorm（QK-Norm），这两者共同帮助稳定训练损失。

下图进一步并排比较了OLMo2和Llama3；正如人们所看到的，除了OLMo2仍然使用传统的多头注意力（MHA）而非全局问答（GQA）之外，两者的架构相对相似。（然而，OLMo2团队三个月后发布了一个使用GQA的32B变体（**OLMo2teamreleaseda32Bvariant**）。

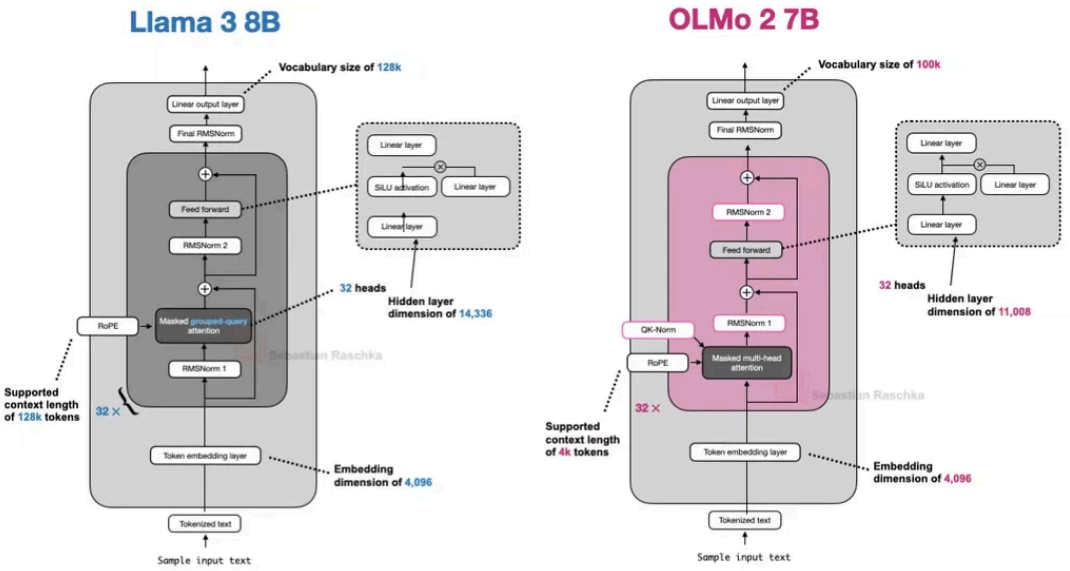


Figure 10: An architecture comparison between Llama 3 and OLMo 2.

3. Gemma3

谷歌的Gemma系列模型一直表现出色，我认为与其他热门模型（如llama系列）相比，它们总是有些被低估。

Gemma的一个显著特点是其相当大的词汇量（以更好地支持多种语言），以及更侧重于270亿参数规模（相对于80亿或700亿参数规模）。但请注意，Gemma2也有较小规模的版本：10亿、40亿和120亿参数。270亿参数规模达到了一个相当好的平衡点：它比80亿参数规模的模型能力更强，但不像700亿参数规模的模型那样资源密集，在我的MacMini上本地运行得很好。

那么，Gemma3中还有什么有趣的地方呢？如前所述，像DeepSeek-V3/R1这样的其他模型采用专家混合（MoE）架构来在推理时降低内存需求，前提是模型大小固定。（MoE方法也被我们稍后要讨论的几个其他模型所使用。）Gemma3采用了不同的“技巧”来降低计算成本，即滑动窗口注意力机制。

3.1滑动窗口注意力

借助滑动窗口注意力（最初在2020年的LongFormer论文中提出（[2004.05150]Longformer:TheLong-DocumentTransformer），并且Gemma2（[2408.00118]Gemma2:ImprovingOpenLanguageModelsataPracticalSize）也使用了该技术），Gemma3团队能够大幅减少键值缓存中的内存需求，如下图所示。

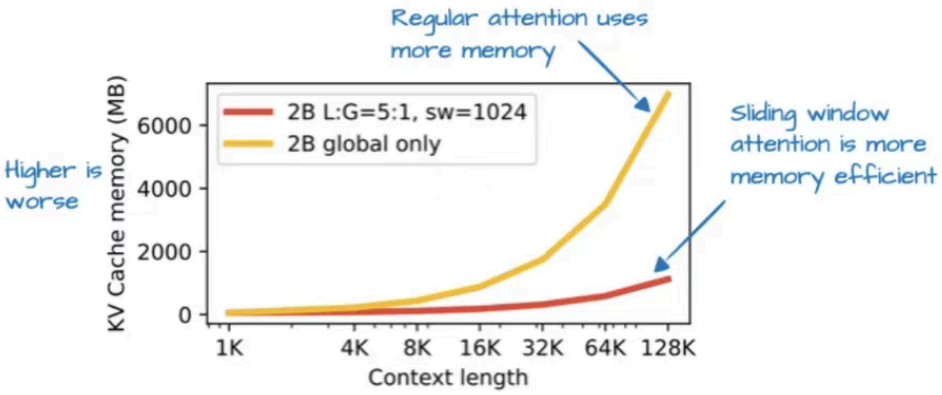


Figure 11: An annotated figure from Gemma 3 paper (https://arxiv.org/abs/2503.19786) showing the KV cache memory savings via sliding window attention.

那么，滑动窗口注意力是什么？如果我们把常规的自注意力机制看作是一种全局注意力机制，因为每个序列元素都可以访问其他每一个序列元素，那么我们可以把滑动窗口注意力看作是一种局部注意力，因为在这里我们限制了当前查询位置周围的上下文大小。如下图所示。

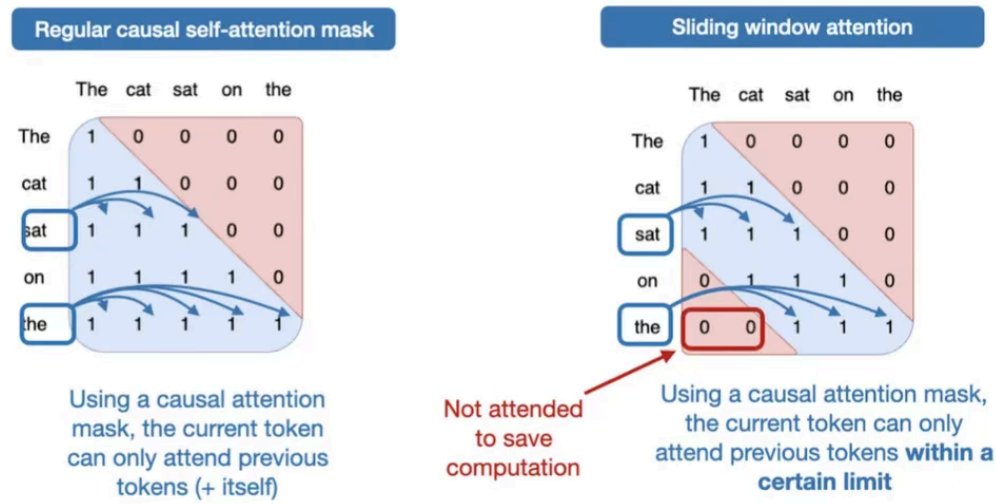


Figure 12: A comparison between regular attention (left) and sliding window attention (right).

请注意，滑动窗口注意力机制既可用于多头注意力（Multi-Head Attention），也可用于分组查询注意力（Grouped-Query Attention）；Gemma3使用的是分组查询注意力。

如上所述，滑动窗口注意力也被称为局部注意力（loca/attention），因为局部窗口围绕当前查询位置并随之移动。相比之下，常规注意力是全局的，因为每个标记可以访问所有其他标记。

现在，如上所述，Gemma2的前身架构也使用了滑动窗口注意力。Gemma3的不同之处在于他们调整了全局（常规）和局部（滑动）注意力的比例。

例如，Gemma2使用了一种混合注意力机制，以1:1的比例结合了滑动窗口（局部）和全局注意力。每个标记可以关注一个包含4k个标记的附近上下文窗口。在Gemma2中，每隔一层使用滑动窗口注意力，而Gemma3现在的比例是5:1，意味着每5个滑动窗口（局部）注意力层只有1个完整的注意力层；此外，滑动窗口大小也从4096（Gemma2）减少到仅1024（Gemma3）。这使模型的重点转向更高效的局部计算。

根据他们的消融研究，如图所示，使用滑动窗口注意力对建模性能的影响最小。



Figure 13: An annotated figure from Gemma 3 paper (https://arxiv.org/abs/2503.19786) showing that sliding window attention has little to no impact on the LLM-generated output perplexity.

虽然滑动窗口注意力是Gemma3架构最显著的特点，但作为对上一节OLMo2的补充，我也想简要回顾一下规范化层的放置。

3.2Gemma3中的规范化层放置

有一个小而有趣的信息值得指出，那就是Gemma3在其分组查询注意力模块周围同时使用RMSNorm进行预规范化和后规范化。

这与Gemma2相似，但仍然值得一提，因为它与（1）原始变压器中使用的后规范化（“注意力就是你所需的一切”），（2）由GPT-2推广并在许多其他架构中使用的预规范化，以及（3）我们之前在OLMo2中看到的后规范化版本不同。

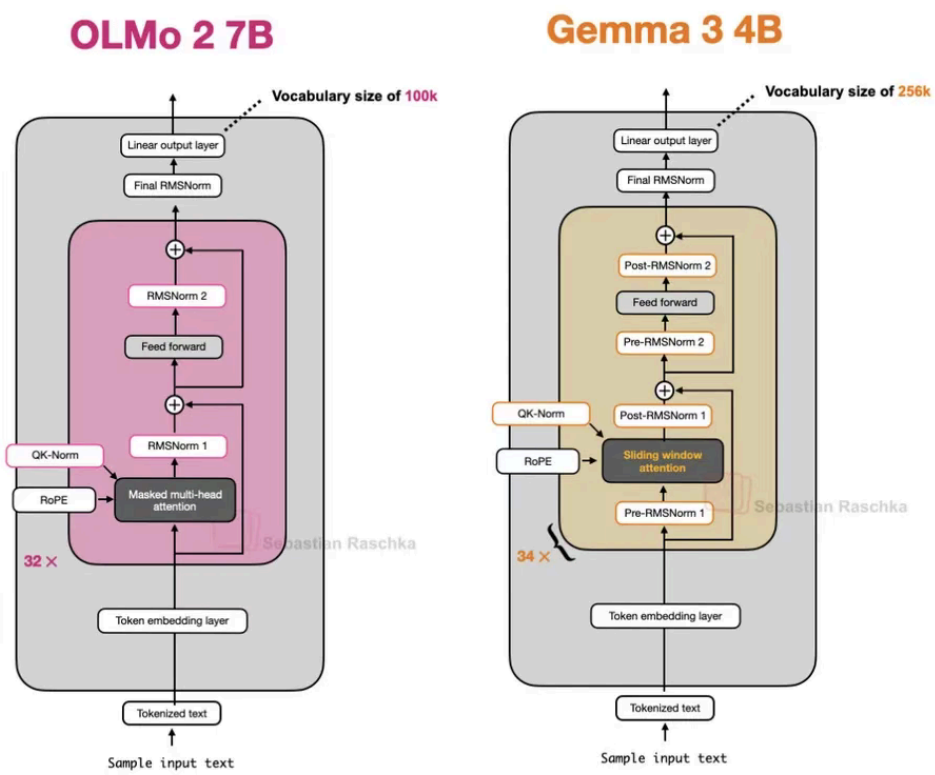


Figure 14: An architecture comparison between OLMo2 and Gemma 3; note the additional normalization layers in Gemma 3.

我认为这种规范化层放置是一种相对直观的方法，因为它兼具了预规范化和后规范化的优点。依我看，增加一些规范化不会有什么坏处。在最坏的情况下，如果额外的规范化是多余的，这将通过冗余增加了一些低效性。不过在实践中，由于RMS规范化在大体上成本相对较低，所以这不应该有任何明显的影响。

3.3Gemma3总结

Gemma3是一款性能良好的开源大型语言模型，在我看来，它在开源社区中有些被低估了。最有趣的部分在于使用滑动窗口注意力来提高效率（将来将其与MoE结合会很有趣）。此外，Gemma3采用了独特的规范化层放置方式，在注意力机制和前馈模块之前和之后都放置了RMS规范化层。

3.4附加内容：Gemma3n

在Gemma3发布几个月后，谷歌分享了Gemma3n（**Gemma3n**），这是一个针对小型设备效率进行了优化的Gemma3模型，旨在能在手机上运行。为了实现更好的效率，Gemma3n的一个变化是所谓的逐层嵌入（PLE）参数层。其核心思想是在GPU内存中仅保留模型参数的一个子集。特定于标记层的嵌入，如文本、音频和视觉模态的嵌入，则按需从CPU或SSD流式传输。

下图展示了PLE的内存节省情况，列出了一个标准Gemma3模型的54.4亿个参数。这可能指的是Gemma3的40亿参数变体。

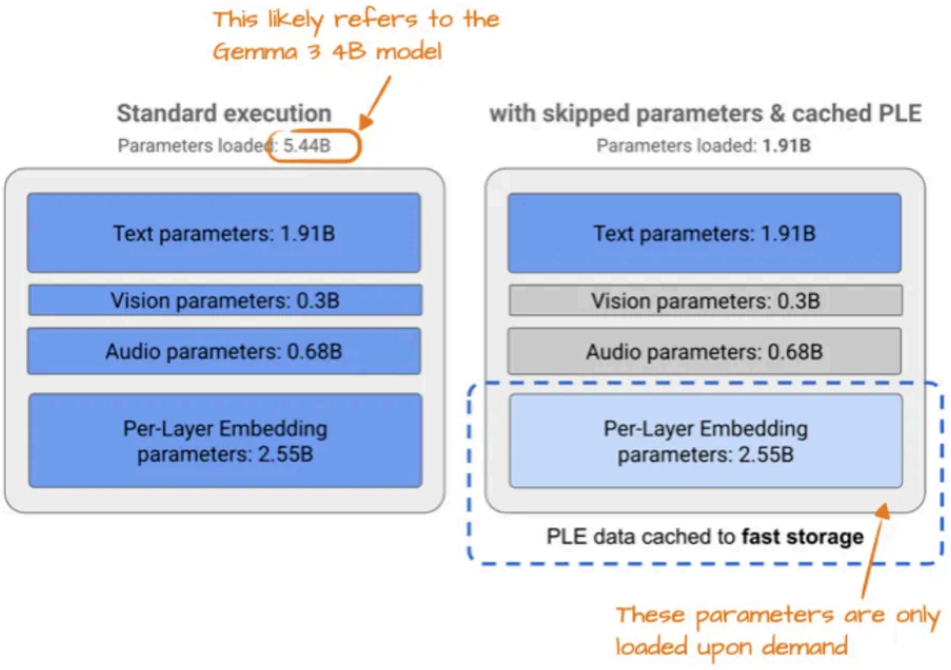


Figure 15: An annotated figure from Google's Gemma 3n blog (https://developers.googleblog.com/en/introducing-gemma-3n/) illustrating the PLE memory savings.

5.44与40亿参数差异的原因是，谷歌在报告大型语言模型（LLM）的参数数量时有一种有趣的方式。他们通常会排除嵌入参数以使模型看起来更小，除了像这种情况，包含它们可以使得模型看起来更大。这并非谷歌独有，因为这种做法已成为该领域的普遍做法。

另一个有趣的技巧是MatFormer[2310.07707]MatFormer:NestedTransformerforElasticInference概念（MatryoshkaTransformer的简称）。例如，Gemma3n使用一个单一的共享LLM（Transformer）架构，可以切割成更小的、独立可用的模型。每个切片都经过训练，使其能够独立运作，因此在推理时，我们只需运行你需要的部分（而不是整个大型模型）。

4. MistralSmall 3.1

MistralSmall3.124B于三月份发布，紧随Gemma3之后，它在多个基准测试中（数学除外）的表现优于Gemma327B，同时还更快。MistralSmall3.1相对于Gemma3推理延迟更低的原因可能归功于其自定义的分词器，以及缩小键值缓存和层数。除此之外，它采用如下图所示的标准架构。

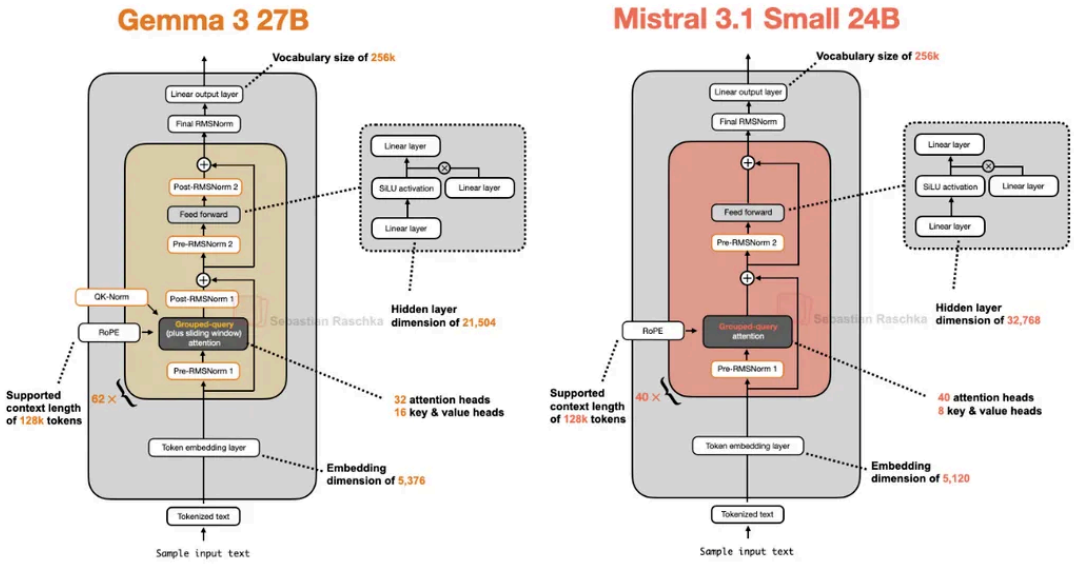


Figure 16: An architecture comparison between Gemma 3 27B and Mistral 3.1 Small 24B.

有趣的是，早期的Mistral模型曾使用滑动窗口注意力机制，但在MistralSmall3.1中似乎放弃了这一做法。因此，由于Mistral使用的是常规的分组查询注意力（Grouped-QueryAttention），而不是像Gemma3中那样的带滑动窗口的分组查询注意力，可能因为能够使用更优化的代码（即Flash Attention）而节省了额外的推理计算资源。例如，我推测虽然滑动窗口注意力减少了内存使用，但并不一定会减少推理延迟，这正是MistralSmall3.1所关注的。

5. Llama4

本文早些时候对专家混合（MoE）的广泛介绍再次得到验证。

Llama4

也采用了MoE方法，并且大体上遵循与DeepSeek-V3非常相似的相对标准架构，如下图所示。（Llama4包括原生多模态支持，类似于Gemma和Mistral等模型。然而，由于本文聚焦于语言建模，我们只关注文本模型。）

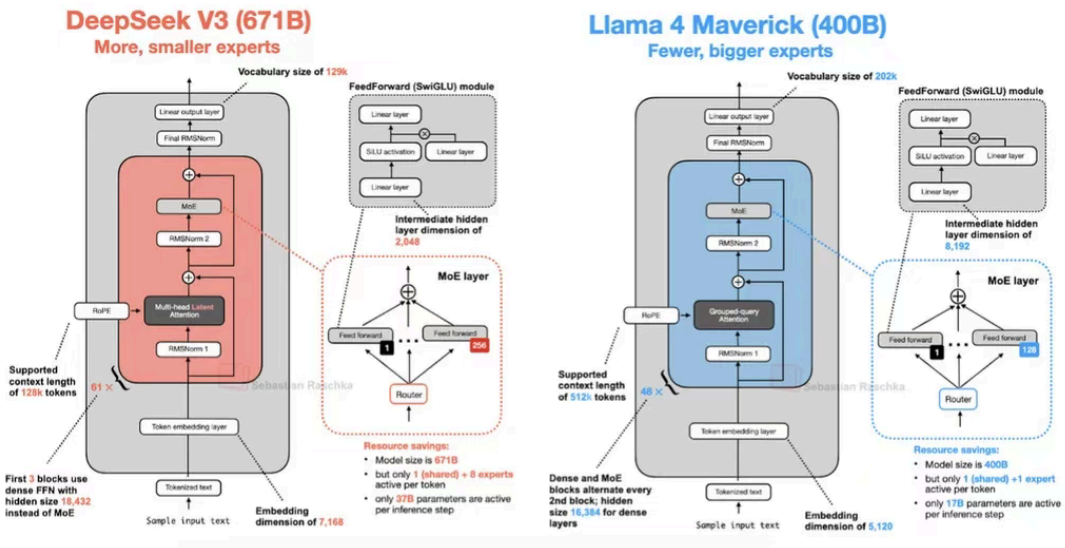


Figure 17: An architecture comparison between DeepSeek V3 (671-billion parameters) and Llama 4 Maverick (400-billion parameters).

图17：DeepSeekV3（拥有6710亿参数）与Llama4Maverick（拥有4000亿参数）之间的架构对比。

尽管Llama4Maverick的架构在整体上与DeepSeek-V3非常相似，但有一些值得重点关注的有趣差异。首先，Llama4使用与其前身类似的组查询注意力机制，而DeepSeek-V3则采用我们在文章开头讨论的多头潜在注意力机制。现在，DeepSeek-V3和Llama4Maverick都是非常庞大的架构，其中DeepSeek-V3的总参数量大约多出68%。然而，DeepSeek-V3有370亿活跃参数，其活跃参数的数量是Llama4Maverick（170亿）的两倍多。

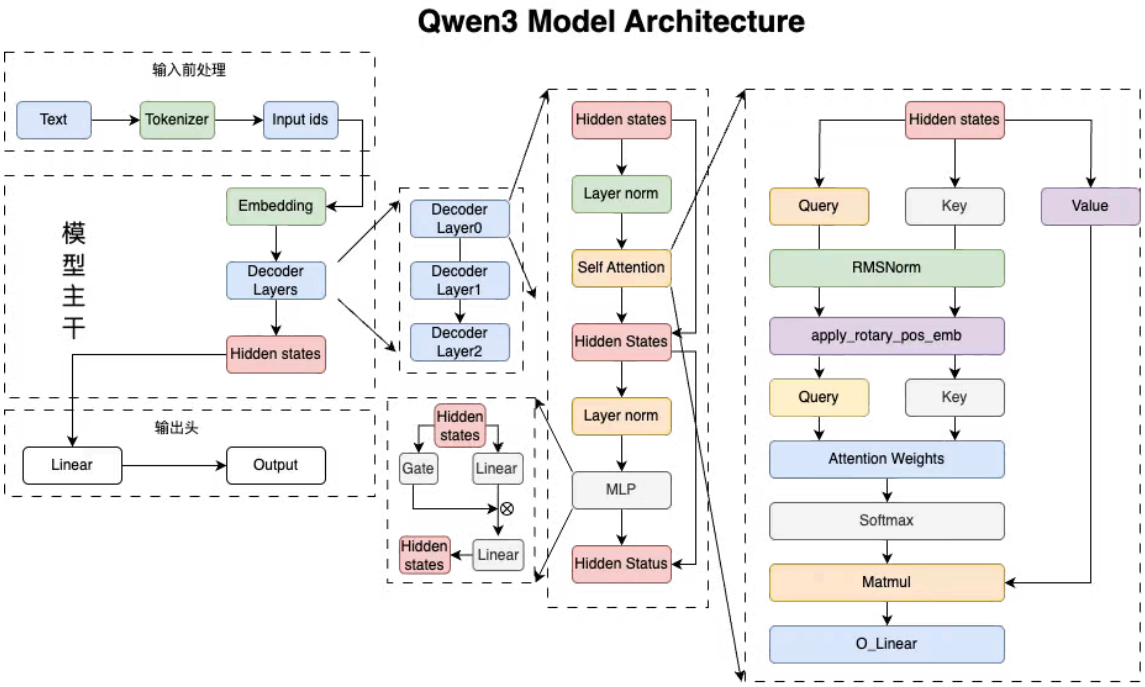
Llama4Maverick采用了一个更为经典的MoE设置，具有较少但更大的专家（每个有8192个隐藏单元的2个活跃专家），相比之下，DeepSeek-V3则有9个活跃专家（每个有2048个隐藏单元）。此外，DeepSeek在每个变换器块中都使用MoE层（前三个除外），而Llama4则在每两个变换器块中交替使用MoE和密集模块。

鉴于架构之间存在许多细微差别，很难确定它们对最终模型性能的确切影响。然而，主要的结论是，到2025年，MoE架构的受欢迎程度显著上升。

6. Qwen3

Qwen团队持续提供高质量的开放权重大型语言模型。当我在2023年的NeurIPS会议上协助指导LLM效率挑战时，我记得获胜的顶尖解决方案都是基于Qwen2的。现在，Qwen3是领先榜单上另一款成功的模型系列，适用于其各个规模类别。共有7个密集模型：0.6B、1.7B、4B、8B、14B和32B。还有2个MoE模型：30B-A3B和235B-A22B。（顺便说一下，“Qwen3”中缺失的空格并非打字错误；我只是想保留Qwen开发者选择的原始拼写。）

与Qwen2比较来看，Qwen3在计算attention的时候增加了对q和k的标准化，在计算q、k矩阵后进行RMSNorm。减少了绝对幅度的学习，而是更关注相对关系，同时降低了计算成本。

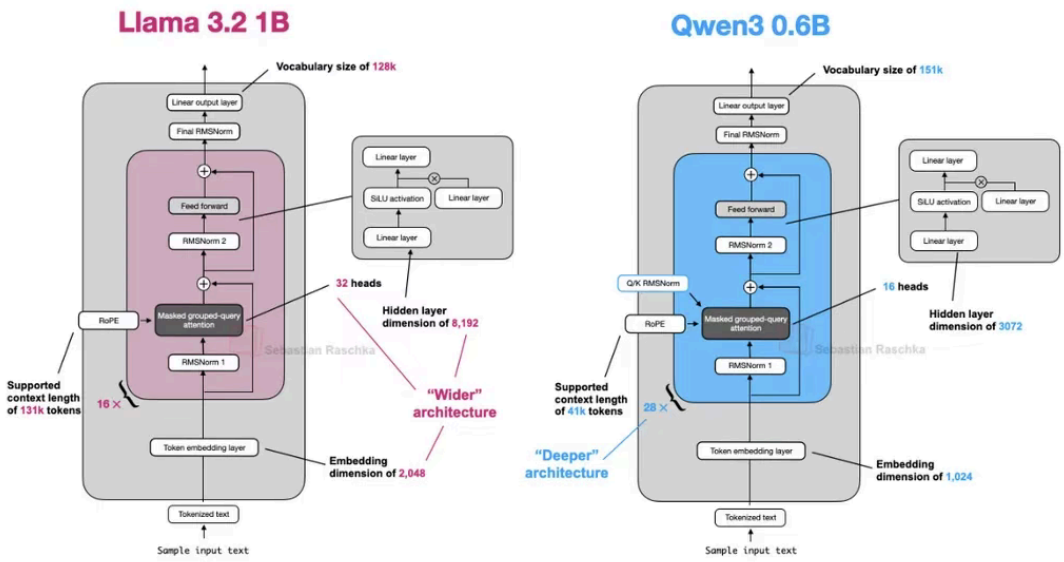


图源：**self-llm/models/Qwen3/01-Qwen3-模型结构解析-Blog.mdatmaster·datawhalechina/self-llm·GitHubself-llm/models/Qwen3/01-Qwen3-模型结构解析-Blog.mdatmaster·datawhalechina/self-llm·GitHub**

6.1Qwen3（Dense）

让我们先讨论一下密集模型的架构。截至本文撰写时，0.6B模型可能是目前最小的开源全权重模型。根据我的个人经验，鉴于其较小的体积，它的表现确实非常好。如果你打算在本地运行它，它具有很高的每秒令牌吞吐量以及较低的内存占用。更重要的是，由于其体积小，它在本地训练起来也很容易（出于教育目的）。

因此，对于我来说，Qwen30.6B已经取代Llama31B用于大多数用途。下面展示了这两种架构的比较。



	Tokens/sec	Memory
Llama3Model 1B	42	2.91 GB
Llama3Model 1B compiled	170	3.12 GB

	Tokens/sec	Memory
Qwen3Model 0.6B	24	1.49 GB
Qwen3Model 0.6B compiled	101	1.99 GB

Figure 18: An architecture comparison between Qwen3 0.6B and Llama 3 1B; notice that Qwen3 is a deeper architecture with more layers, whereas Llama 3 is a wider architecture with more attention heads.

如果您对无需依赖外部第三方大型语言模型库的可读性强的Qwen3实现感兴趣，我最近从零开始用纯PyTorch实现了Qwen3。

上图中的计算性能数据基于我在A100GPU上运行的从零开始的PyTorch实现。正如所见，Qwen3的内存占用较小，因为其架构本身较小，同时也使用了较小的隐藏层和较少的注意力头。上图中的计算性能数据基于我的从零开始的每秒PyTorch生成令牌数）。

6.2Qwen3（MoE）

如前所述，Qwen3也有两种MoE变体：30B-A3B和235B-A22B。为什么有些架构，如Qwen3，会有常规（密集）和MoE（稀疏）两种变体呢？

正如本文开头所提到的，MoE变体有助于降低大型基础模型的推理成本。提供密集和MoE两种版本可以让用户根据他们的目标和限制灵活选择。密集模型通常更易于微调、部署和优化，适用于各种硬件。另一方面，MoE模型则针对推理扩展进行了优化。例如，在固定的推理预算下，它们可以在不相应增加推理成本的情况下实现更高的整体模型容量（即训练期间由于模型更大而吸收的知识）。

通过发布这两种类型，Qwen3系列可以支持更广泛的使用场景：密集模型用于鲁棒性、简单性和微调；MoE模型用于大规模高效服务。为了总结这一部分，让我们看看Qwen3 235B-A22B（注意A22B代表“22B个活动参数”）到DeepSeek-V3，后者的活动参数几乎是前者的两倍（370亿个）。

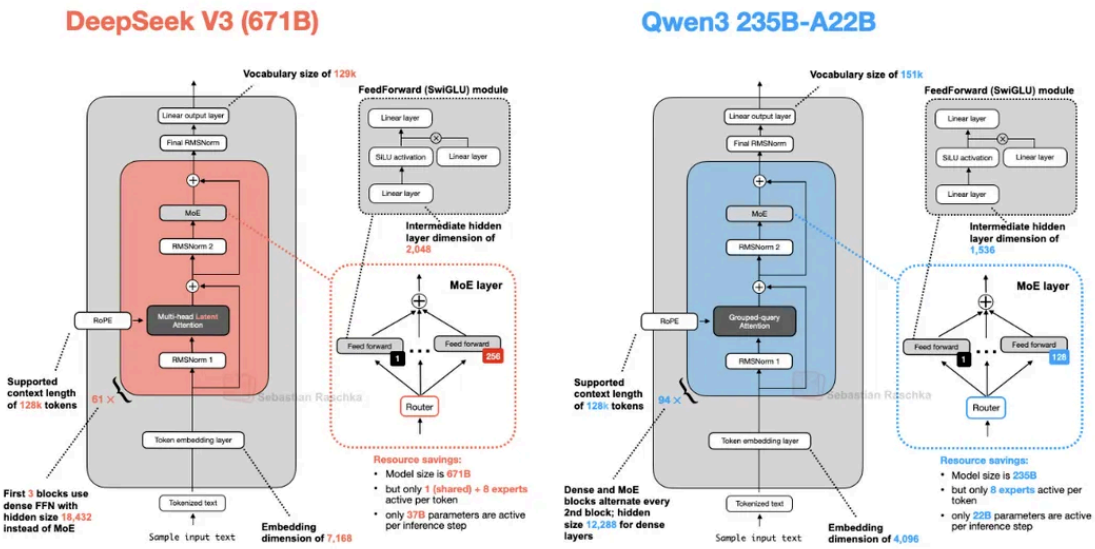
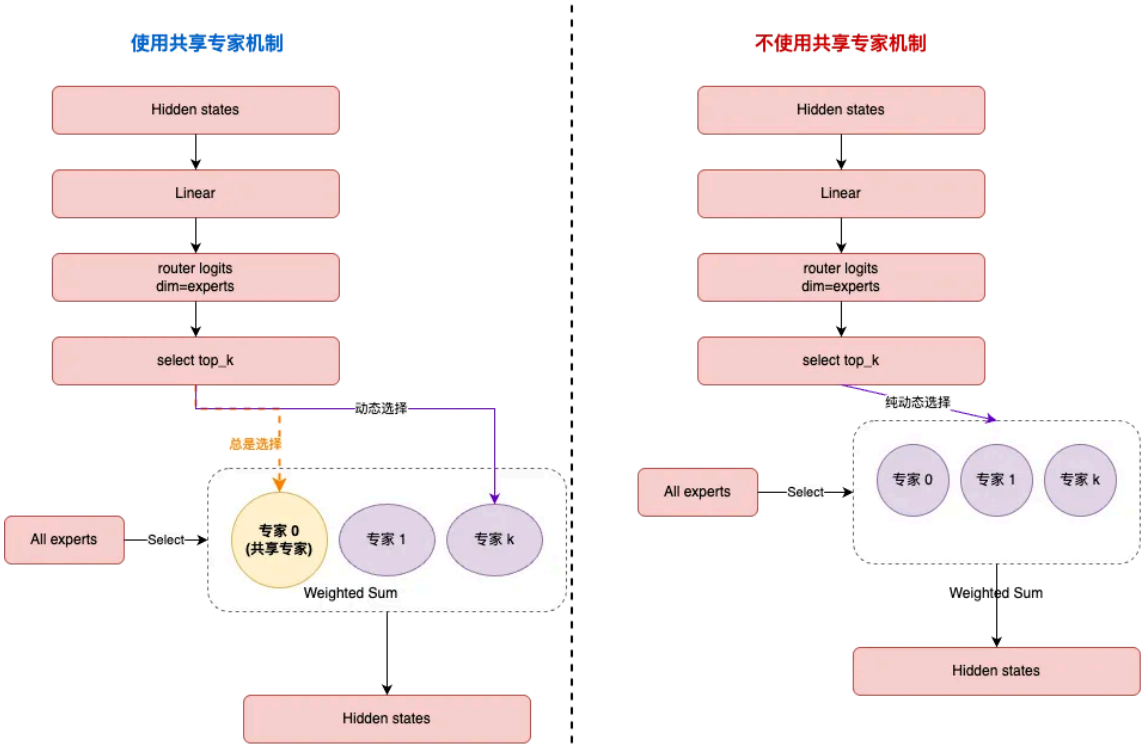


Figure 19: An architecture comparison between DeepSeek-V3 and Qwen3 235B-A22B.

如上图所示，DeepSeek-V3和Qwen3 235B-A22B架构非常相似。然而值得注意的是，**Qwen3模型不再使用共享专家**（早期的Qwen模型，例如**Qwen2.5-Max: Exploring the Intelligence of Large-scale MoEModel| Qwen**确实使用了共享专家）。



不幸的是，Qwen3团队并没有透露他们为何放弃使用共享专家的原因。如果让我猜测，可能是因为在专家数量从**2个（在Qwen2.5-MoE中）增加到8个（在Qwen3中）**时，对于他们的设置而言，训练稳定性并不需要共享专家。然后他们通过仅使用8个而不是8+1个专家来节省额外的计算/内存成本。（然而，这并没有解释为什么DeepSeek-V3仍然保留他们的共享专家。）

可能是效果差不多，但这样实现更简单？或者是想试试看不同的设置？

MOE怎么计算？

另外说一下，7月中下旬的时候，Qwen放出了Qwen3系列模型的更新。7月21号开始，Qwen陆续放出更新版本，主要更新上下文长度、工具调度能力等：Qwen3-235B-A22B-Instruct-2507（256K上下文）、Qwen3-Coder-480B-A35B-Instruct（256K上下文，（通过YaRN可扩展至1M））、Qwen3-235B-A22B-Thinking-2507（256K上下文）、Qwen3-30B-A3B-Instruct-2507

●**Qwen3-235B-A22B-Instruct-2507**：在GPQA（知识）、AIME25（数学）、LiveCodeBench（编程）等评测中超越Kimi-K2等模型。支持256K上下文长度，增强了多语言长尾知识覆盖，优化了用户偏好对齐和长文本建模。

●**Qwen3-Coder-480B-A35B-Instruct（7.22）**：在代码生成、工具调用等方面性能提升，如在SWE-benchVerified等基准测试中取得较好成绩，支持256K上下文长度，增强了代码理解和生成能力。【后训练强化】引入了CodeRL和Long-HorizonRL技术，以提高代码执行成功率和解决复杂软件工程问题的能力

●**Qwen3-235B-A22B-Thinking-2507**：在推理性能和通用能力上实现飞跃，可比肩顶尖闭源模型。在逻辑推理、数学运算等推理任务中表现大幅提升，支持256K长上下文理解。

●**Qwen3-30B-A3B-Instruct-2507**：是Qwen3-30B-A3B的高质量指令微调版本，以非思考模式运行。相较于前代，其推理能力（AIME25）提升了183.8%，对齐能力（Arena-Hardv2）提升了178.2%，长文本处理能力由128K提升至256K。在多语言长尾知识覆盖、主观与开放任务的文本质量、代码生成、数学计算、工具使用等通用能力上也表现出全面进步，更注重稳态输出与一致性，适合稳定生产环境部署。

阿里开源了其最强代码模型：Qwen3-Coder-480B-A35B-Instruct，性能媲美ClaudeSonnet4-阿里云开发者社区



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

7. SmolLM3

SmolLM3可能没有像本文中涵盖的其他大型语言模型那样受欢迎，但我觉得将其作为一个有趣的模型包括进来还是很好的，因为它在相对较小且方便的三十亿参数模型大小下提供了非常好的建模性能，该模型大小介于17亿和40亿参数的Qwen3模型之间，如下图所示。

此外，它还分享了许多训练细节，类似于OLMo。

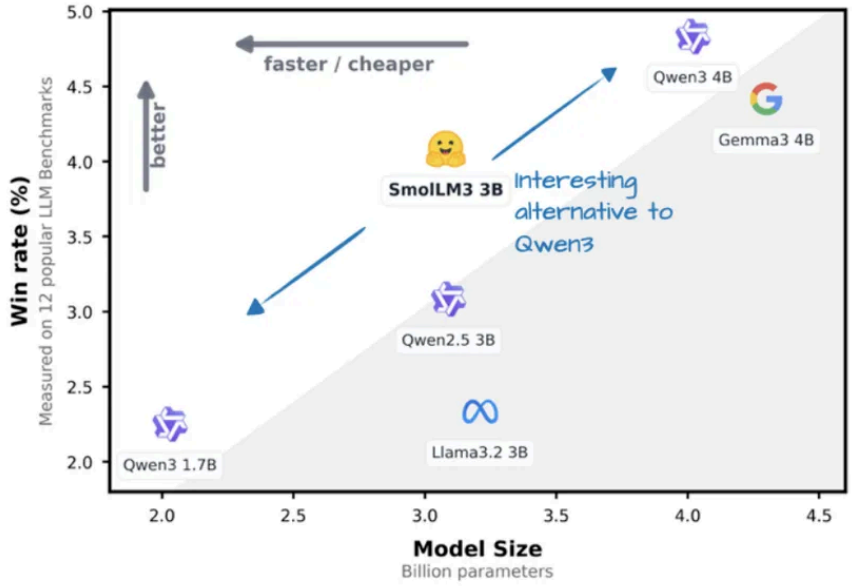


Figure 20: An annotated figure from the SmolLM3 announcement post, https://huggingface.co/blog/smollm3, comparing the SmolLM3 win rate to Qwen3 1.7B and 4B as well as Llama 3 3B and Gemma 3 4B.

如下方的架构比较图所示，SmolLM3架构看起来相当标准。不过，也许最有趣的方面是它使用了NoPE（无位置嵌入）。

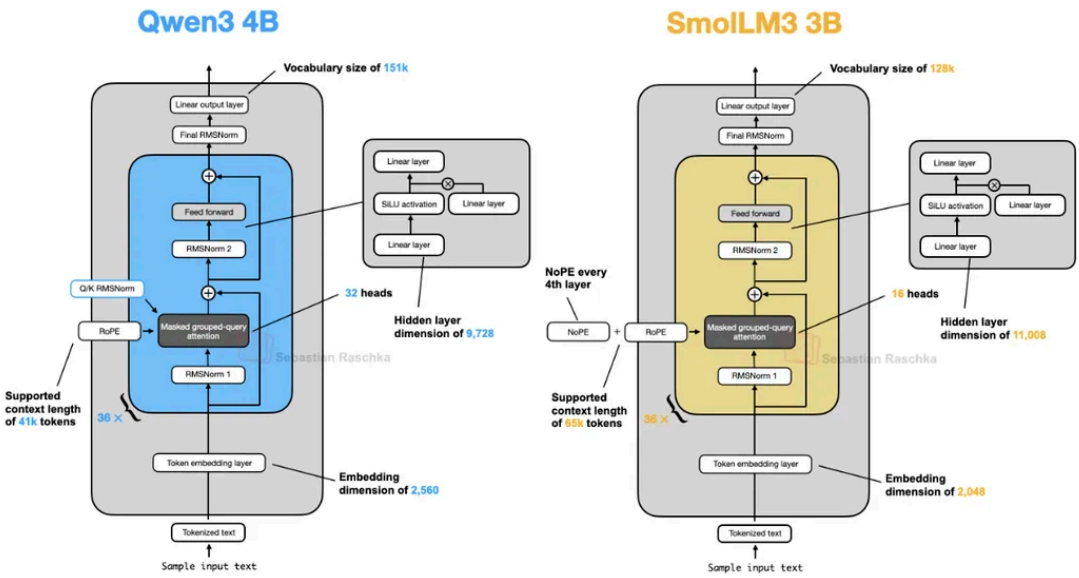


Figure 21: A side-by-side architecture comparison between Qwen3 4B and SmolLM3 3B.

7.1无位置嵌入（NoPE）

在大型语言模型（LLM）的背景下，无位置嵌入是一个较旧的概念，可以追溯到2023年的一篇论文《[2305.19466]TheImpactofPositionalEncodingonLengthGeneralizationinTransformers》。该概念旨在移除显式的位置信息注入（如通过早期GPT架构中的经典绝对位置嵌入层或现今的RoPE）。

在基于变压器的LLM中，位置编码通常必不可少，因为自注意力机制独立于顺序处理标记。绝对位置嵌入通过添加一个额外的嵌入层来解决这一问题，该层向标记嵌入中添加信息。

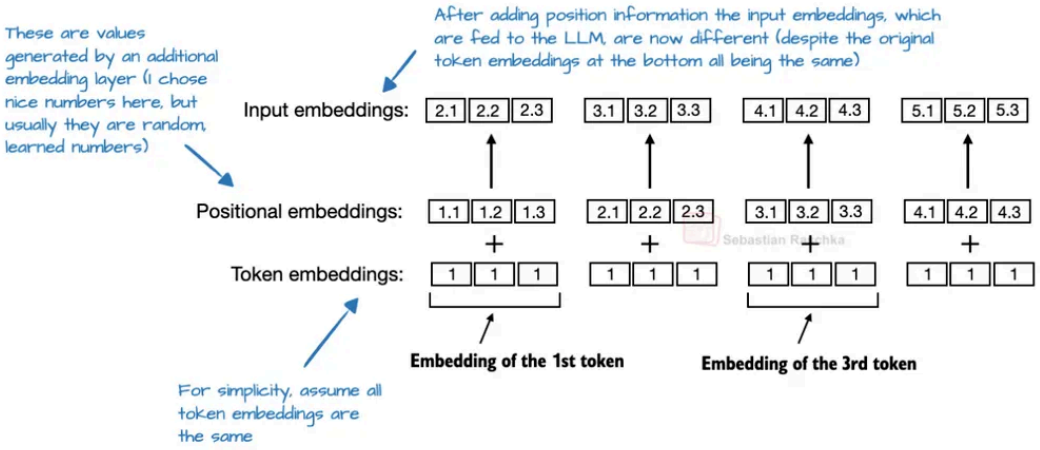


Figure 22: A modified figure from my Build A Large Language Model (From Scratch) book (https://www.amazon.com/Build-Large-Language-Model-Scratch/dp/1633437167) illustrating absolute positional embeddings.

另一方面，RoPE通过相对于它们的标记位置旋转查询向量和键向量来解决这个问题。然而，在NoPE层中，根本不会添加此类位置信号：不是固定的、不是学习的、也不是相对的。什么都没有。尽管没有位置嵌入，但由于因果注意力掩码，模型仍然知道哪些标记在前。这个掩码防止每个标记关注未来的标记。因此，位于位置t的标记只能看到位置≤t的标记，这样就保留了自回归排序。

所以，虽然没有明确添加位置信息，但在模型的结构中仍然隐含有一种方向感，而在常规基于梯度下降的训练中，大型语言模型（LLM）如果发现它有利于优化目标，就能够学会利用它。（欲了解更多信息，请查看NoPE论文中的定理。）总的来说，**[2305.19466]TheImpactofPositionalEncodingonLengthGeneralizationinTransformers**不仅发现不需要注入位置信息，而且还发现NoPE具有更好的长度泛化能力，这意味着随着序列长度的增加，大型语言模型的答题表现下降得较少，如下图所示。

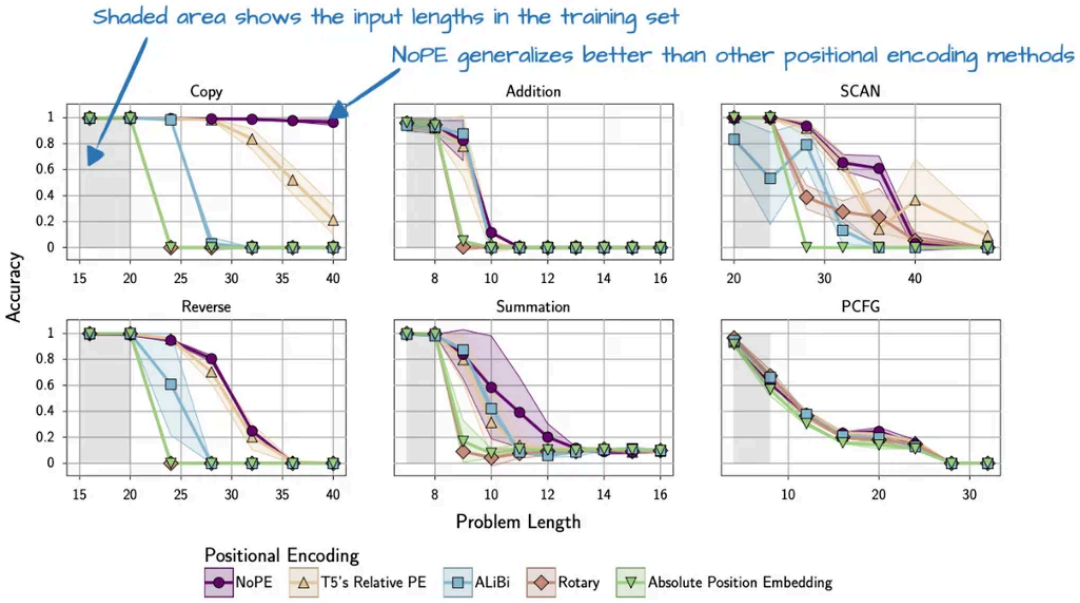


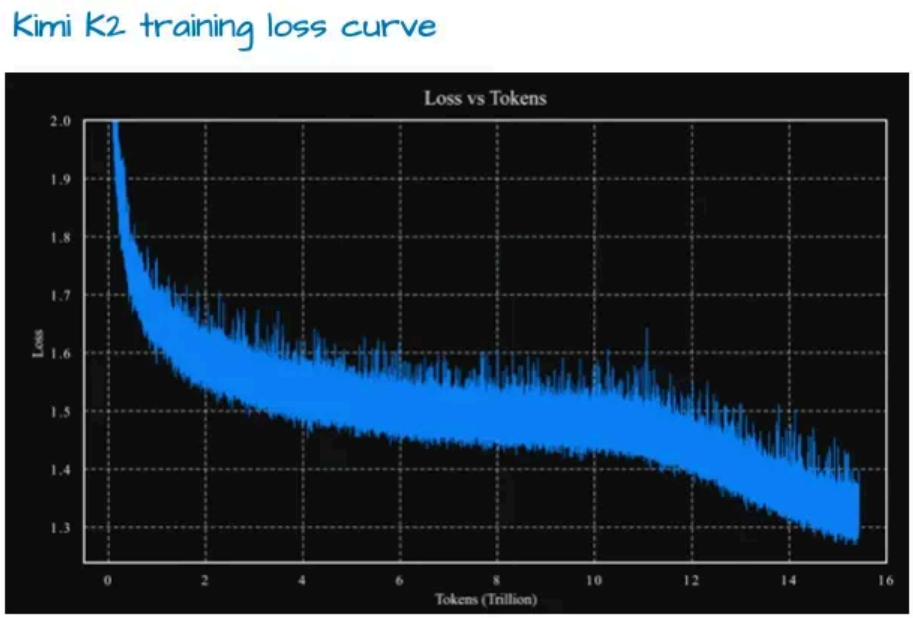
Figure 23: An annotated figure from the NoPE paper (https://arxiv.org/abs/2305.19466) showing better length generalization with NoPE.

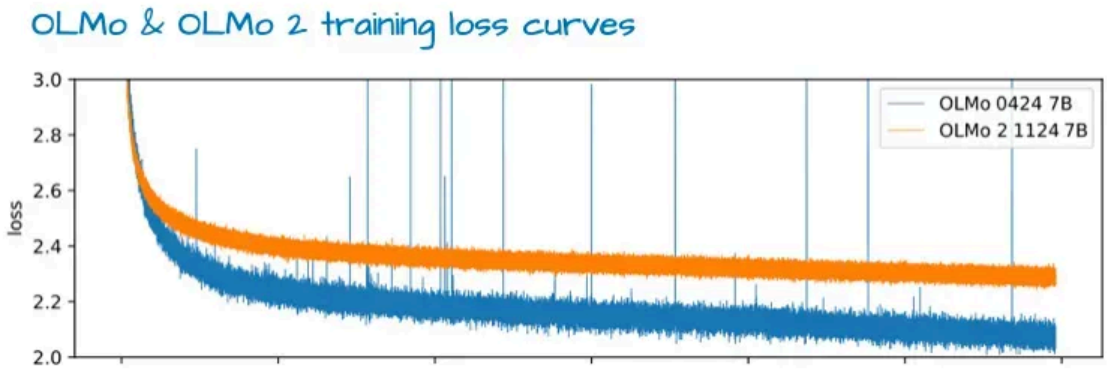
8. Kimi2

KimiK2:OpenAgenticIntelligence最近在人工智能社区引起了轰动，因为它是一个开放权重模型，且性能极为出色。根据基准测试，它的表现与谷歌的Gemini、Anthropic的Claude以及OpenAI的ChatGPT等最佳专有模型不相上下。

技术报告：**Kimi-K2/tech_report.pdfatmain·MoonshotAI/Kimi-K2·GitHub**

值得注意的是，它使用了相对较新的Muon优化器变体，而非AdamW。据我所知，这是首次将Muon用于任何如此规模的量产模型（此前，仅被证明可以扩展到160亿参数规模）。这导致了非常漂亮的训练损失曲线，可能帮助该模型在上述基准测试中脱颖而出。虽然人们评论说损失异常平滑（由于缺乏尖峰），但我认为它并非异常平滑（例如，请参见下图中的OLMo2损失曲线；此外，梯度的L2范数可能是跟踪训练稳定性的更好指标）。然而，值得注意的是损失曲线的衰减情况非常好。然而，正如本文引言中所提到的，训练方法论是另一个话题。





该模型本身有1万亿参数，确实令人印象深刻。截至本文撰写时，它可能是这一代最大的语言模型（考虑到Llama4Behemoth尚未发布、专有语言模型不计入以及谷歌的1.6万亿SwitchTransformer是采用不同代的编码器-解码器架构）。

同时，它也回到了起点，因为Kimi2使用了我们在文章开头提到的DeepSeek-V3架构，只不过他们将模型做得更大了，如下图所示。

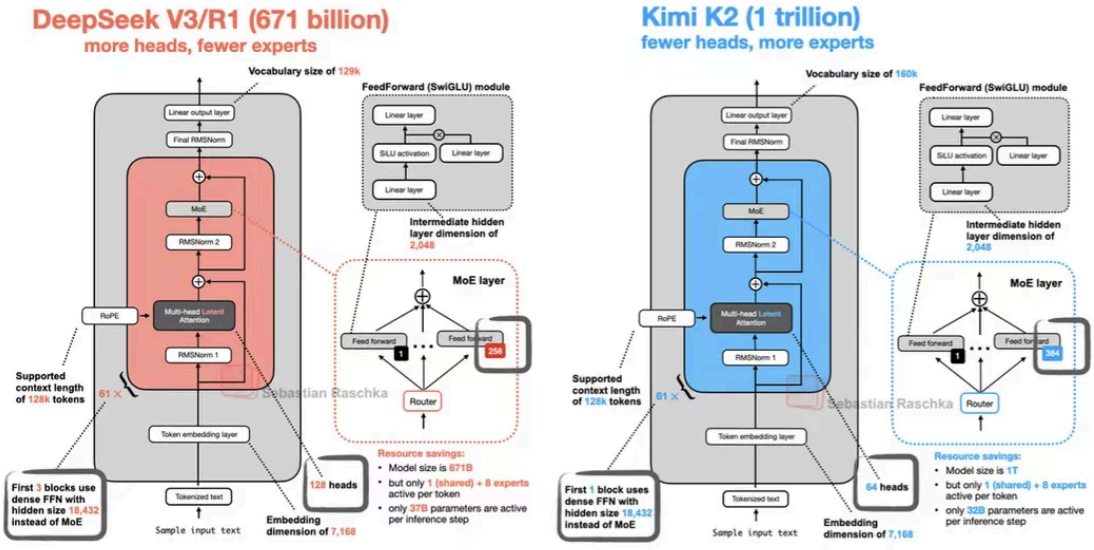


Figure 25: An architecture comparison between DeepSeek V3 and Kimi K2.

如上图所示，Kimi2.5基本上与DeepSeekV3相同，除了它在MoE模块中使用了更多专家，在多头潜在注意力（MLA）模块中使用了较少的头。

Kimi2 并 非 毫 无 缘 由 地 出 现 。 早 期 在《**[2501.12599]Kimik1.5:ScalingReinforcementLearningwithLLMs**》论文中讨论的Kimi1.5模型也令人印象深刻。然而，它运气不佳，DeepSeekR1模型的论文恰好在同一天，即1月22日发布。而且据我所知，Kimi1.5的权重从未公开分享。

因此，很可能KimiK2团队吸取了这些教训，并在DeepSeekR2发布之前，将KimiK2作为一个开放权重模型公开。目前来说，K2模型的效果是整体实测相对较好的（但也有它的问题存在）。

9. GLM4.5/4.6

[2508.06471]GLM-4.5:Agentic,Reasoning,andCoding(ARC)FoundationModels

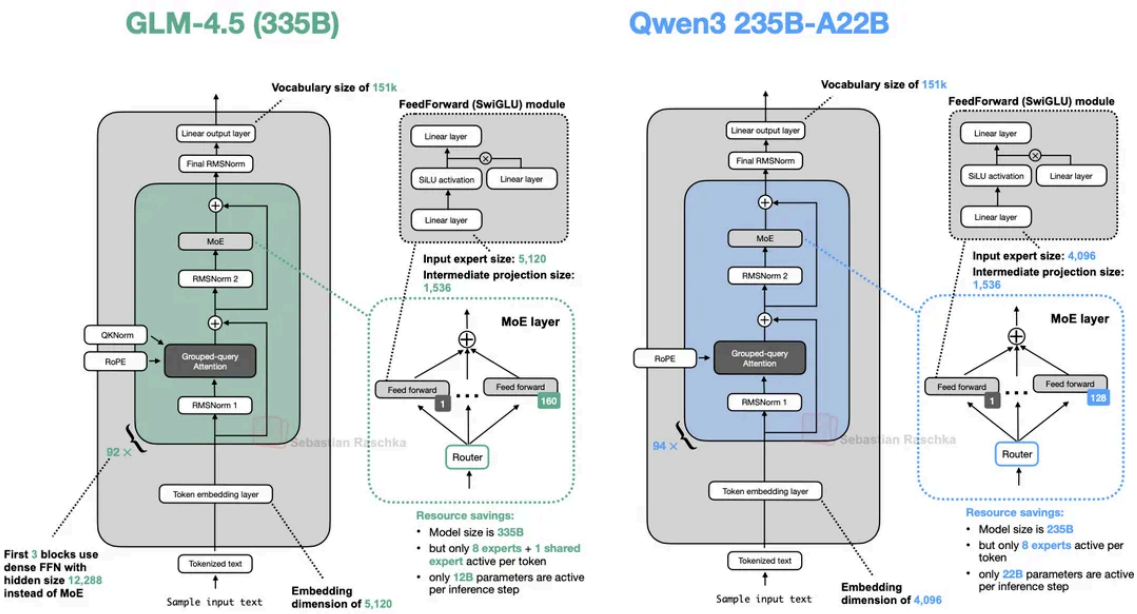
GLM-4.5的核心突破在于首次在单个模型中**原生融合**了三大关键能力（ARC），以衡量其通用人工智能（AGI）水平：

- **A(Agentic)智能体能力**：模型与外部工具（如搜索引擎、代码编辑器、API）及现实世界进行交互，以完成多步复杂任务的能力。

- **R(Reasoning)复杂推理能力**：解决需要多步逻辑推导的问题，尤其在数学、科学等领域的复杂问题求解能力。

- **C(Coding)高级编程能力**：应对真实世界的软件工程任务，例如修复GitHub issue、全栈应用开发等。

其设计遵循“第一性原理”，旨在**不损失原有通用能力的前提下，融合更多通用智能能力**，标志着大模型从“知识库”向“求解器”的演进。



4.5版本官方公布了相关技术论文，目前4.6版本暂未从官方渠道找到相关技术报告。4.5的模型架构跟其他主流开源模型类似，都采用了MOE训练。GLM4.5包含两个主要版本：

模型版本	总参数量	激活参数量	网络层数	专家数量
GLM-4.5	3550亿	320亿	89层	160个
GLM-4.5-Air	1060亿	120亿	45层	128个

主要技术特点包括：

●**深而非宽的设计**：借鉴了DeepSeek-V3的部分思想，但减少了模型宽度（隐藏维度为5120），增加了模型深度（最多89层）。这种设计被认为对提升推理能力更有效。

●**注意力机制优化**：-采用**分组查询注意力（Grouped-Query Attention,GQA）**，提升长上下文处理的效率。-使用多达**96个注意力头**（约为Llama3的1.5倍），虽未显著降低训练损失，但在MMLU、BBH等推理基准上表现更好。-引入**RoPE（旋转位置编码）**并调整其基频至1,000,000，以支持长达128K的上下文窗口。-应用**QK-Norm**稳定注意力logits的范围，防止训练不稳定。

●**高效的MoE路由**：采用**无损平衡路由（loss-free balance routing）**和**Sigmoid门控机制**，确保专家负载均衡。

●**推测式解码支持**：在**多Token预测（MTP）**层中加入MoE层，以支持推理阶段的推测式解码，从而提升生成速度。

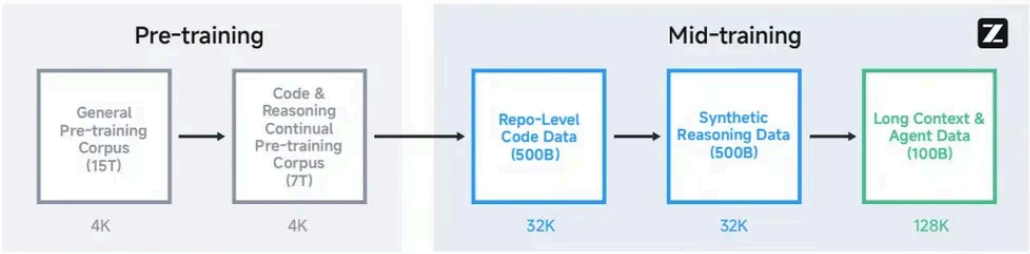
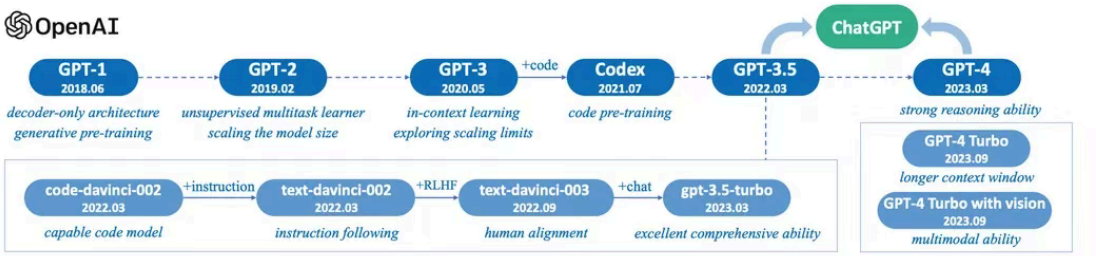


Figure : Pre-training and mid-training stages for GLM-4.5. We adapt a multi-stage training recipe and extend the sequence length from 4K to 128K.

训练流程上，GLM-4.5的训练流程严谨而复杂，可分为三个核心阶段（预训练、中期训练和后训练），总计使用了约**23万亿（23T）Token**的多源、高质量数据。预训练阶段主要让模型掌握通用的语言、知识和基础代码能力，采用精细化数据策略，对高质量网页、代码及科教内容进行上采样，并引入语义去重和填充式训练等方法。中期训练聚焦于专业能力强化和上下文扩展，通过整合仓库级代码数据、合成推理语料以及智能体交互轨迹，将上下文长度逐步扩展到了128K。后训练阶段先将基础模型分化为推理、智能体和通用聊天三个独立专家进行专项训练，再通过自蒸馏技术将专家能力统一整合到一个主模型中，最终形成具备“思考模式”和“非思考模式”的混合推理架构，既可深度分析也能快速响应。

三、非开源组核心模型2025年表现

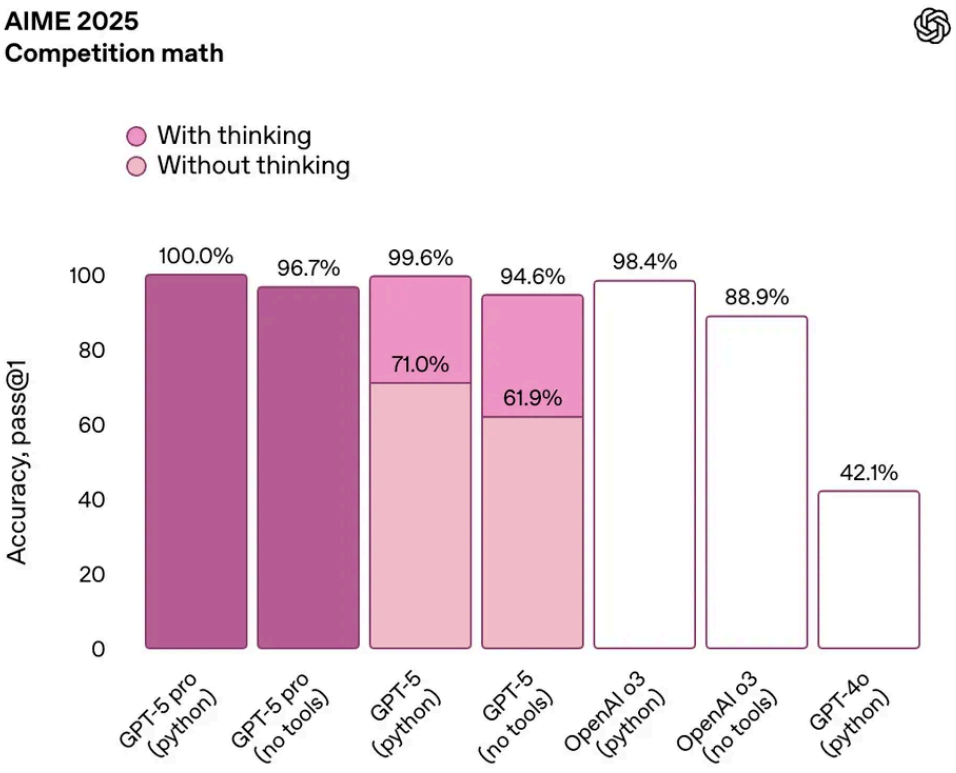
10.GPT5



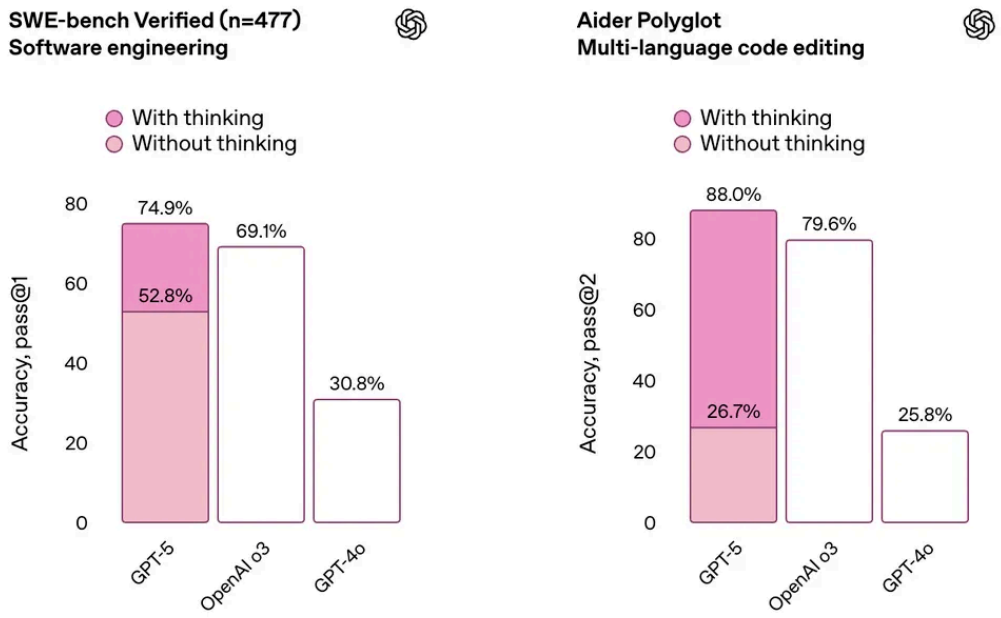
IntroducingGPT-5|OpenAI

核心带来了统一路由架构。更多是产品架构的改进而非技术突破。猜测在模型训练的道路上可能是遇到了瓶颈，跷跷板问题难以打破，难以同时提高所有任务的性能。不同模型发挥所长也符合一定的思考定式，简单问题我们不希望过多思考依靠直觉即可，而复杂问题我们希望能在深入思考推理之后再得到答案，也有一点点思考快与慢的意思了。

使用上，目前GPT5面向所有用户开放，Plus会员可获得更多使用量，Pro会员则可访问GPT-5Pro版本。评估侧，它**在数学（AIME2025无需工具测试得分94.6%）、真实世界编码（SWE-benchVerified得分74.9%，AiderPolyglot得分88%）、多模态理解（MMMU得分84.2%）和健康（HealthBenchHard得分46.2%）方面均创下了新的最高水平——这些提升在日常使用中得到了充分体现。凭借GPT-5pro的扩展推理能力，该模型还在GPQA**上创下了新的最高水平，无需工具测试得分高达88.4%。



编码



所有 SWE-bench 评估运行均使用已在我们的内部基础设施上验证过的 n=477 个已验证任务的固定子集。

关于幻觉问题，OpenAI自己评估GPT5编造信息的比例比上一代降低60%。

11.Claude4

Claude系列无疑是目前在编程领域是最强的模型，写代码爽到飞起。因闭源的缘故，技术相关的报告不多。

- **ClaudeSonnet4**：在AmazonBedrock中，其上下文窗口已从20万Token扩展至100万Token，能处理更长文本内容，在大规模代码分析和文档综合分析等方面更高效。

- **Claude4.1Opus**：2025年8月5日发布，在编程能力上有显著提升，在SWE-benchVerified中得分74.5%，Terminal-bench中得分43.3%，分别超越了部分竞品。同时，它在安全方面也有升级，拒绝违反政策请求的比例有所提高

- ClaudeSonnet4.5：2025年9月29日发布，代表了AI编码和代理功能的重大演进，使其成为复杂软件开发和长期任务执行的领先解决方案。

- 在编码任务方面树立了新的行业标杆。它在SWE-benchVerified测试（衡量真实世界中GitHub问题解决率的指标）中达到了82.0%的准确率，超越了GPT-5（74.9%）和Gemini2.5Pro（67%）等竞争对手。在OSWorld基准测试（真实世界中的计算机任务执行）中，它的准确率达到了61.4%，比其前代产品（Sonnet4的42.2%）提升了近50%。该模型还展现出卓越的长期任务处理能力，能够持续专注于复杂的多步骤项目超过30小时（例如，构建一个包含11,000多行代码的Slack风格聊天应用）。

●ClaudeSonnet4.5在通用AI任务方面也表现出色：1）推理与数学：它在金融、法律、医疗和STEM领域表现出色，例如，在研究生水平推理（GPQADiamond）中的准确率高达83.4%，在多语言问答（MMMLU）中的准确率高达89.1%。2）计算机应用：在OSWorld基准测试中，其执行实际计算机任务（例如，浏览浏览器、填写表单）的能力显著提升，使其在终端用户应用中更加灵活。

● ClaudeSonnet4.5 可通过 Claude 官网、API（模型 ID：claude-sonnet-4-5）以及AmazonBedrock等第三方平台获取。值得一提的是，定价与Sonnet4保持不变：每百万输入代币3美元，每百万输出代币15美元，为现有用户带来更高价值。

●ClaudeHaiku4.5：2025年10月16日发布，轻量级模型，定位为高性价比、低延迟的实时任务解决方案。性能比肩中端：编程性能与5个月前的中端模型Sonnet4相当（SWE-bench约73.3%）。速度与成本：推理速度比Sonnet4快2倍以上，成本仅为其三分之一。

参考文献

1）**Kimi-K2/tech_report.pdfatmain·MoonshotAI/Kimi-K2·GitHub**

2）**IntroducingGPT-5|OpenAI**

3）**TheBigLLMArchitectureComparison**

4）**Qwen**

5）**[2505.09388]Qwen3TechnicalReport**

6）**[2412.19437]DeepSeek-V3TechnicalReport**

7）**[2501.12948]DeepSeek-R1:IncentivizingReasoningCapabilityinLLMsviaReinforcementLearning**

8）**[2502.11089]NativeSparseAttention:Hardware-AlignedandNativelyTrainableSparseAttention**

9）**Kimi-K2/tech_report.pdfatmain·MoonshotAI/Kimi-K2·GitHub**

10）**[2508.06471]GLM-4.5:Agentic,Reasoning,andCoding(ARC)FoundationModels**

11）**[2501.11873]DemonsintheDetail:OnImplementingLoadBalancingLossforTrainingSpecializedMixture-of-ExpertModels**

12）**[2503.02356]EfficientLongContextFine-tuningwithChunkFlow**



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容

公众号



阅读 6355

留言

写留言

还没有留言，来写首评